

zVSAM V2 Design and Logic Manual V2.0

Table of Contents

[Introduction](#)

[Copyright](#)

[Acknowledgements](#)

[Terminology](#)

[Compatibility](#)

[zVSAM compatibility with IBM VSAM](#)

[zVSAM V2 compatibility with zVSAM V1](#)

[API for Assembler and zCOBOL programs](#)

[ACB-based interfaces](#)

[ACB macro](#)

[GENCB ACB macro](#)

[MODCB ACB macro](#)

[SHOWCB ACB macro](#)

[TESTCB ACB macro](#)

[RPL-based interfaces](#)

[RPL macro](#)

[GENCB RPL Macro](#)

[MODCB RPL Macro](#)

[SHOWCB RPL Macro](#)

[TESTCB RPL Macro](#)

[EXLST macro](#)

[GENCB EXLST macro](#)

[MODCB EXLST macro](#)

[SHOWCB EXLST macro](#)

[TESTCB EXLST macro](#)

[OPEN macro](#)

[CLOSE macro](#)

[POINT Macro](#)

[GET Macro](#)

[PUT Macro](#)

[ERASE Macro](#)

[CHECK Macro](#)

[ENDREQ Macro](#)

[VERIFY Macro](#)

Catalog management

Physical structure of the files

Basic Concepts

Files, Blocks, Records

Cluster types and Cluster Components

Record Formats

Concept of Fixed length records stored in blocks

Concept of Fixed length Segmented records stored in blocks

Concept of Variable length records stored in blocks

Concept of Variable length Segmented records stored in blocks

File Structure

Physical files

Structure of physical files

Prefix Block

Spacemap Blocks

Data Blocks

Index Blocks

Raw Blocks

Structure of ESDS

Structure of KSDS

Structure of RRDS

Structure of LDS

Free Blocks

Block Structures

Block Header Structure

Prefix Block chain summary

Example 1: Unsegmented Data Block structure

Example 2: Segmented Data Block structure

Example 3: Index Block with single level structure

Example 4: Index Block with two-level structure

Block Footer Structure

Record Pointer List Structure

Free Space

Prefix Area Structure

Counters Area Structure

Counters Area values

Spacemap Area Structure

Spacemap bit values

Fixed Record Structure

Variable Record Structure

Fixed Segmented Record Structure

Variable Segmented Record Structure

Logical processes for ACB-based requests

ACB

GENCB ACB

MODCB ACB

SHOWCB ACB

TESTCB ACB

EXLST

GENCB EXLST

MODCB EXLST

SHOWCB EXLST

TESTCB EXLST

OPEN

OPEN macro logic

OPEN execution logic

CLOSE

Logical processes for RPL-based requests

RPL macro

GENCB RPL macro

MODCB RPL macro

SHOWCB RPL macro

TESTCB RPL macro

POINT macro

GET macro

PUT macro

ERASE macro

CHECK macro

ENDREQ macro

VERIFY macro

Addenda

API for ACB-based interfaces

ACB Macro parameters

GENCB ACB macro parameters

MODCB ACB macro parameters

SHOWCB ACB macro parameters

TESTCB ACB macro parameters

EXLST macro parameters

GENCB EXLST macro parameters

MODCB EXLST macro parameters

SHOWCB EXLST macro parameters

TESTCB EXLST macro parameters

OPEN macro parameters

CLOSE macro parameters

zACB description

zEXLST description

CBMR description

CBMR description – body for ACB

CBMR description – body for EXLST

API for RPL-based interfaces

RPL macro parameters

GENCB RPL macro parameters

MODCB RPL macro parameters

SHOWCB RPL macro parameters

TESTCB RPL macro parameters

POINT macro parameters

GET macro parameters

PUT macro parameters

ERASE macro parameters

CHECK macro parameters

ENDREQ macro parameters

VERIFY macro parameters

zRPL description

CBMR description – body for RPL

List of changes

Introduction

This document describes the structure of the zVSAM component of the z390 assembler and emulator. It consists of the following parts:

[A description of the structure of the interfaces used](#)

[A description of the structure of the files](#)

[A description of the logical processes that implement ACB-based requests](#)

[A description of the logical processes that implement RPL-based requests](#)

[Addenda](#)

Copyright Notice

This document Copyright 2018 – z390 development team

z390 is free software; its associated documentation is equally free. You can redistribute and/or modify both software and documentation under the terms of the GNU General Public License as published by the Free Software Foundation; either version 2 of the License, or (at your option) any later version

z390 is distributed in the hope that it will be useful, but WITHOUT ANY WARRANTY; without even the implied warranty of MERCHANTABILITY or FITNESS FOR A PARTICULAR PURPOSE. See the GNU General Public License for more details.

You should have received a copy of the GNU General Public License along with z390; if not, write to the Free Software Foundation, Inc., 59 Temple Place, Suite 330, Boston, MA 02111-1307 USA.

Acknowledgements

z390's purpose is to provide a source-level compatible assembler, linker and run-time emulation engine for IBM's High-Level Assembler environment. By extension, the zVSAM component described in this document also aims at providing source-level compatibility. As a consequence, all source-level interfaces necessarily mimic the IBM-provided (and IBM-copyrighted) interfaces as described by IBM in publicly accessible documents.

The logic and implementation behind these interfaces, however, was developed independently from IBM and is the product of the joint efforts of our team of volunteer developers.

Source-level compatibility is a primary goal not only for z390 and zVSAM, but also for other z390 components such as zCOBOL and zCICS.

All IBM publications and software we refer to in this document are copyright IBM Corporation with no exception.

The drawings in this document have been made using the site draw.io.

As part of the open source for z390 the xml documents describing these drawings are available with every distribution of z390 that contains this document.

Terminology

The reader is assumed to have at least some familiarity with IBM VSAM, to the extent that the following acronyms and terms are understood:

-ACB	Access Control Block
-AIX	Alternate Index
-CBMR	Control Block Modification Request
-CI	Control Interval
-ESDS	Entry Sequenced Data Set
-IBM	International Business Machines Corp., USA
-KSDS	Key Sequenced Data Set
-LDS	Linear Data Set
-Path	Access to a base cluster, usually through an AIX
-RBA	Relative Byte Address
-RDW	Record Descriptor Word in IBM-defined format
-RLF	Record Length Field – z390 equivalent of RDW
-RPL	Request Parameter List
-RRDS	Relative Record Data Set
-RRN	Relative Record Number
-SPX	Segment Prefix
-VSAM	Virtual Storage Access Method
-XRBA	Extended Relative Byte Address
-XLRA	Extended Logical Record Address
-zACB	z390 equivalent of the ACB
-zEXLST	z390 equivalent of the EXLST
-zRPL	z390 equivalent of the RPL

In this document we also use the following terms. The ones that are used by IBM as well, are intended to have the same meaning they do in IBM manuals

Area	a section of storage with a defined layout, depending on the type of Area
Block	zVSAM equivalent of a Control Interval
Cluster	a set of files that logically belong together
Component	either a data component or an index component of a cluster
File	a single file as seen by the hosting operating system
Foxes	a value consisting of all high-values i.e. a value of all X'FF' bytes.
List	a structure holding items that are linked together by pointers
Segment	a portion of a record in a spanned dataset
Segmented	a record that has been split into segments in a spanned dataset
Spanned	an attribute of a dataset that allows records to be split into segments
Sphere	a cluster and all associated AIXs
Table	a structure holding items that are physically adjacent

Compatibility

As this document relates to zVSAM V2, there are two type of compatibility we need to consider. On the one hand we have designed zVSAM to be compatible with IBM VSAM. And on the other hand we need to consider compatibility with z390's zVSAM V1 – the prior implementation of zVSAM in the z390 environment.

zVSAM compatibility with IBM VSAM

Our z390 implementation of zVSAM V2 is intended to be source-level compatible with IBM VSAM. This has the following consequences:

1. IBM VSAM documentation with regards to macros and interfaces applies to zVSAM with the exception of parameters and options not supported by zVSAM. Where zVSAM differs in behaviour this is noted in this document. Please refer to the macro descriptions in section II for details.
2. Control Blocks (such as ACB, RPL and some others) are not compatible. zVSAM has its own structures. A side effect of this may be that a program's assembled object code may be different in size than on your IBM operating system. On rare occasions you may need an additional base register when porting your program either way.
3. As a rule of thumb, a program using VSAM can be ported to z390 and should be able to assemble, link and run without modification – provided it uses only the VSAM features and options that zVSAM supports. And provided the program does not run out of addressability due to different control block lengths.

zVSAM V2 compatibility with zVSAM V1

The user of z390's zVSAM component should be aware that zVSAM V2 as described in this document is not compatible with the pre-existing zVSAM V1. We – the development team – apologize for the inconvenience this may cause.

We have taken the following measures to facilitate the transition from zVSAM V1 to zVSAM V2:

- 1) We have introduced a new z390 option: ZVSAM which indicates which version of zVSAM you want z390 to use. It takes the following forms:
ZVSAM(0) – zVSAM usage is disallowed
ZVSAM(1) – zVSAM V1 is enabled, zVSAM V2 is disabled
ZVSAM(2) – zVSAM V2 is enabled, zVSAM V1 is disabled
For max. compatibility the default is set to ZVSAM(1).
The default will be changed to ZVSAM(2) in a future release of z390.
- 2) To convert your zVSAM V1 clusters to zVSAM V2 you'll have to take the following steps:
 - unload the existing data from their clusters using REPRO
 - reload your data from your unload files, using ZREPROFor details on how to use REPRO, please refer to the "z390_VSAM_User_Guide"
For details on how to use ZREPRO, please refer to the "z390_zVSAM_zREPRO_User_Guide"
- 3) For zVSAM V1 and zVSAM v2 there are distinct macro libraries.
To use the correct zVSAM maclib, specify the correct version in your maclib concatenation

API for Assembler and zCOBOL programs

ACB-based interfaces

The ACB is the primary interface for operations at the cluster level. Each cluster is represented by an ACB.

- ACB.Open
- ACB.Close

The ACB interface consists of an ACB control block, possibly an Exit list Control Block, and a set of macros to manage and manipulate the ACB and EXLST control blocks. These macros can be used in your assembler programs. For zCOBOL and/or other higher-level languages, these macros will be generated from specifications for the files as appropriate in the host language's syntax.

The following macros are provided for assembler programs:

- ACB
- GENCB ACB
- MODCB ACB
- SHOWCB ACB
- TESTCB ACB

Note: The ACB macro defines a statically allocated ACB. This macro is primarily intended for use in non-re-entrant programs. GENCB ACB should be used to create an ACB in dynamically acquired storage, or in private static storage. MODCB ACB can be used to modify a pre-existing ACB, whereas SHOWCB ACB can be used to query specific fields of an ACB and TESTCB ACB can be used to validate specific fields of an ACB.

- EXLST
- GENCB EXLST
- MODCB EXLST
- SHOWB EXLST
- TESTCB EXLST

Note: The EXLST macro defines a statically allocated EXLST. This macro is primarily intended for use in non-re-entrant programs. GENCB EXLST should be used to create an EXLST in dynamically acquired storage, or in private static storage. MODCB EXLST can be used to modify a pre-existing EXLST, whereas SHOWCB EXLST can be used to query specific fields of an EXLST and TESTCB EXLST can be used to validate specific fields of an EXLST.

- OPEN
- CLOSE

Note: Open and Close macros can be used to open and close either sequential files represented by a DCB and/or zVSAM files represented by an ACB.

A description of these interfaces as implemented for z390 and zVSAM is detailed in the next chapters

ACB Macro

The ACB macro will generate an ACB and initialize it according to the parameters specified on the macro invocation.

The structure and layout of the generated ACB are not part of the interface and are therefore not shown in this chapter. Direct access to subfields in the ACB is discouraged. Use SHOWCB ACB, TESTCB ACB and/or MODCB ACB to inspect, test, and/or modify the ACB's content.

All keywords on the ACB macro are optional. Before the cluster is opened, all ACB values can be modified using MODCB ACB, or by changing the ACB directly. The latter is not recommended, as it is not guaranteed to be portable or compatible with future versions of zVSAM.

The table below shows how the ACB macro can be coded.

Opcode	Operand	Remarks
[label] ACB	[AM=VSAM]	Designates this ACB as a zVSAM ACB
	[DDNAME=ddname]	DDNAME: name of an environment variable in the host OS holding the name of the cluster to be processed
	[PASSWD=ptr]	Pointer to password for the cluster
	[EXLST=ptr]	Pointer to EXLST structure
	[MACRF=(keywd_list)]	List of keywords specifying processing options. See table below for valid keywords
	[BUFSP=nr]	Max amount of storage (in bytes) to use for buffers
	[BUFND=nr]	Number of data buffers to allocate for this ACB
	[BUFNI=nr]	Number of index buffers to allocate for this ACB
	[RMODE31=keyword]	Indicates whether buffers and/or control blocks can be allocated above the line
	[STRNO=nr]	Number of concurrent requests allowable for this ACB
	[BSTRNO=nr]	Beginning number of concurrent requests allowable for this ACB
	[MAREA=ptr]	Not supported – future option. Keyword is flagged as ignored with a warning message (Level 4 MNOTE)
	[MLEN=nr]	Not supported – future option. Keyword is flagged as ignored with a warning message (Level 4 MNOTE)
	[RLSREAD=keyword]	Not supported – future option. Keyword is flagged as ignored with a warning message (Level 4 MNOTE)
	[SHRPOOL=nr]	LSR shared pool number

Supported options for the MACRF parameter are listed below:

Keyword subset	Keyword	Remarks
[ADR <u>KEY</u>]	ADR	Addressed access to ESDS
	KEY	Keyed access to KSDS or RRDS
	CNV	Not supported – future option. Keyword is flagged as ignored with a warning message (Level 4 MNOTE)
[DFR <u>NDE</u>]	DFR	Allow writes to be deferred
	NDF	Disallow deferred writes
[DIR <u>SEQ</u> <u>SKP</u>	DIR	Direct access to RRDS
	SEQ	Sequential access to ESDS, KSDS or RRDS
	SKP	Skip sequential access to KSDS or RRDS
[<u>IN</u> <u>OUT</u>]	IN	Read only access for ESDS, KSDS or RRDS
	OUT	Both read and write access for ESDS, KSDS or RRDS
[<u>NIS</u> <u>SIS</u>]	NIS	Normal Insert Strategy for KSDS
	SIS	Sequential Insert Strategy
[<u>NRM</u> <u>AIX</u>]	NRM	DDNAME indicates cluster to be processed
	AIX	DDNAME indicates an AIX to be processed as a path into its base cluster
[<u>NRS</u> <u>RST</u>]		Not supported – future option. Keyword is flagged as ignored with a warning message (Level 4 MNOTE)
[<u>LSR</u> <u>GSR</u> <u>NSR</u> <u>RLS</u>]		Local, Global or no Shared Buffers. RLS is not supported
[<u>NUB</u> / <u>UBF</u>]		Not supported – future option. Keyword is flagged as ignored with a warning message (Level 4 MNOTE)
[<u>CFX</u> / <u>NFX</u>]		Not supported – future option. Keyword is flagged as ignored with a warning message (Level 4 MNOTE)
[<u>DDN</u> / <u>DSN</u>]		Not supported – future option. Keyword is flagged as ignored with a warning message (Level 4 MNOTE)
[<u>ICI</u> / <u>NCI</u>]		Not supported – future option. Keyword is flagged as ignored with a warning message (Level 4 MNOTE)
[<u>LEW</u> / <u>NLW</u>]		Not supported – future option. Keyword is flagged as ignored with a warning message (Level 4 MNOTE)

DDNAME=

DDNAME is required before open is executed. If DDNAME is not supplied on the ACB macro, the label used on the ACB macro is used as DDNAME. If neither is specified, a proper value must be supplied by using MODCB ACB.

In zVSAM the DDNAME is to hold the name of an environment variable in the host OS. This variable in turn should contain the path and qualified filename of the cluster to be opened. The qualifier is the name of an environment variable in the host OS and is the path to the assembled catalog. For more information on zVSAM catalogs, please refer to the "z390_zVSAM_Catalog_User_Guide".

With the exception of the DDNAME parameter explained above, all supported parameters are implemented compatibly with IBM's VSAM implementation. For details, please refer to the relevant IBM manual. For ease of access a short summary can be found in the addenda.

GENCB ACB macro

The GENCB macro with ACB=acb will generate or manipulate ACBs and initialize or change them according to the parameters specified on the macro invocation. It is for this reason that all supported parameters and keywords of the ACB macro (as described above) are supported on the GENCB macro when ACB= is specified.

The structure and layout of the generated ACB are not part of the interface and are therefore not shown in this chapter. Direct access to subfields in the ACB is discouraged. Use SHOWCB ACB, TESTCB ACB and/or MODCB ACB to inspect, test, and/or modify the ACB's content.

Likewise, the structure and layout of the CBMR that zVSAM uses to transfer the GENCB request to the CBMR handler are not part of the interface and are therefore not shown in this chapter. Direct access to subfields in the CBMR is strongly discouraged.

The GENCB ACB macro can be coded as follows:

Opcode	Operand	Remarks
[label] GENCB	ACB=acb	Instructs GENCB to generate 1 or more ACBs
	[AM=VSAM]	Optional, no other values allowed
	[COPIES=nr]	The number of identical ACBs to generate
	[WAREA=addr]	The work area where the ACBs are to be constructed
	[LENGTH=nr]	Length of the work area in bytes
	[LOC=keyword]	Where GENCB is to allocate dynamically acquired storage - if needed
	[other]	Any parameter supported on the ACB macro
	[MF=I or omitted]	Use standard form of GENCB ACB
	[MF=L/MF=(L,addr,[label])]	Use list form of GENCB ACB
	[MF=(E,addr)]	Use execute form of GENCB ACB
	[MF=(G,addr,[label])]	Use generate form of GENCB ACB

All supported parameters are implemented compatibly with IBM's VSAM implementation. For details, please refer to the relevant IBM manual.

For ease of access a short summary can be found in the addenda.

MODCB ACB macro

The MODCB macro with ACB=addr will modify an ACB according to the parameters specified on the macro invocation. It is for this reason that all parameters and keywords of the ACB macro (as described above) are supported on the MODCB macro when ACB=addr is specified.

The structure and layout of the affected ACB are not part of the interface and are therefore not shown in this chapter. Direct access to subfields in the ACB is discouraged. Use SHOWCB ACB, TESTCB ACB and/or MODCB ACB to inspect, test, and/or modify the ACB's content.

Likewise, the structure and layout of the CBMR that zVSAM uses to transfer the MODCB request to the CBMR handler are not part of the interface and are therefore not shown in this chapter. Direct access to subfields in the CBMR is strongly discouraged.

The MODCB ACB macro can be coded as follows:

Opcode	Operand	Remarks
[label] MODCB	ACB=address	Points MODCB to the ACB to be modified
	[other]	Any parameter supported on the ACB macro
	[MF=I or omitted]	Use standard form of MODCB ACB
	[MF=L/MF=(L,addr,[label])]	Use list form of MODCB ACB
	[MF=(E,addr)]	Use execute form of MODCB ACB
	[MF=(G,addr,[label])]	Use generate form of MODCB ACB

All supported parameters are implemented compatibly with IBM's VSAM implementation. For details, please refer to the relevant IBM manual.

For ease of access a short summary can be found in the addenda.

SHOWCB ACB macro

The SHOWCB macro with ACB=addr will return ACB-related fields according to the parameters specified on the macro invocation in the order they are specified.

The structure and layout of the affected ACB are not part of the interface and are therefore not shown in this chapter. Direct access to subfields in the ACB is discouraged. Use SHOWCB ACB, TESTCB ACB and/or MODCB ACB to inspect, test, and/or modify the ACB's content.

Likewise, the structure and layout of the CBMR that zVSAM uses to transfer the SHOWCB request to the CBMR handler are not part of the interface and are therefore not shown in this chapter. Direct access to subfields in the CBMR is strongly discouraged.

The SHOWCB ACB macro can be coded as follows:

Opcode	Operand	Remarks
[label] SHOWCB	ACB=address	Points SHOWCB to the ACB to be queried
	AREA=addr	Address of return area
	LENGTH=nr	Size of return area in bytes
	[OBJECT=DATA/INDEX]	For KSDS: select data or index component
	FIELDS=(keywd_list)	List of keywords indicating which fields to return
	[MF=I or omitted]	Use standard form of SHOWCB ACB
	[MF=L/MF=(L,addr,[label])]	Use list form of SHOWCB ACB
	[MF=(E,addr)]	Use execute form of SHOWCB ACB
	[MF=(G,addr,[label])]	Use generate form of SHOWCB ACB

Supported options for the FIELDS parameter are listed below:

Keyword	Length	Remarks
ACBLEN	4	Size of ACB in bytes
AVSPAC	4	Available space in component
BFRND	4	Nr of buffer hits for component (No I/O needed to satisfy read request)
BSTRNO	4	Initial nr of strings
BUFND	4	Nr of data buffers specified in ACB
BUFNI	4	Nr of index buffers specified in ACB
BUFNO	4	Number of buffers in use for component
BUFRDS	4	I/O count in buffers
BUFSP	4	Buffer space in bytes specified in ACB

Keyword	Length	Remarks
CDTASIZE	8	Size of the component or High Water Mark?
CINV	4	Block size for component
DDNAME	8	DDNAME specified in ACB
ENDRBA	4	PFXHLRA, recalculated to RBA value of Block's last byte
ERROR	4	Return code from last open/close operation
EXLST	4	Ptr to EXLST, foxes if no EXLST applies
FS	4	Nr of free blocks per 100 blocks in the component. Derived from PFXFRBLK and PFXFRINT values.
HALCRBA	4	Highest valid XLRA in the component, recalculated to an RBA value
KEYLEN	4	Length of key field
LEVEL	8	Address (4 bytes) and length (4 bytes) of field containing zVSAM version number
LOKEY	8	Ptr (4 bytes) to lowest key in the cluster + length (4 bytes) of key
LRECL	4	Maximum record length; foxes if in excess of 4GB
MAREA	4	Ptr to message area, foxes if not relevant
MLEN	4	Length of message area, foxes if not relevant
NCIS	4	Nr of Block splits in the data component. Foxes for index.
NDELRL	4	Nr of deleted records from data component. Foxes for index.
NEXCP	4	Nr of I/O requests for the component
NEXT	4	Nr of extents to the physical file. Foxes.
NINSR	4	Nr of records inserted for the data component. Foxes for index.
NIXL	4	Nr of index levels for index component. Foxes for data component.
NLOGR	4	Nr of records in the component
NRETR	4	Nr of records retrieved from the data component. Foxes for index.
NSSS	4	Nr of control area splits. Foxes.
NUIW	4	Nr of implicit write operations.
NUPDR	4	Nr of updated records in the component
PASSWD	4	Ptr to password, consisting of length (1 byte, binary) followed by actual password value
RELEASE	8	Address (4 bytes) and length (4 bytes) of field containing zVSAM version number
RKP	4	Relative Key Position, offset of key within logical record
SDTASIZE	8	Component size in bytes
SHRPOOL	4	SHRPOOL number
STMST	8	System timestamp of last close
STRMAX	4	Max nr of concurrently active strings
STRNO	4	Max nr of allocated strings

Keyword	Length	Remarks
UIW	4	Nr of explicit writes for component
XAVCSPAC	8	AVCSPAC when value may exceed 4GB
XENDRBA	8	ENDRBA when value may exceed 4GB
XHALCRBA	8	HALCRBA when value may exceed 4GB

All supported parameters and keywords are implemented compatibly with IBM's VSAM implementation. For details, please refer to the relevant IBM manual.

For ease of access a short summary can be found in the addenda.

TESTCB ACB macro

The TESTCB macro with ACB=addr will test ACB-related fields according to the parameters specified on the macro invocation. Only a single test can be specified on each TESTCB invocation. TESTCB returns a PSW condition code of 8=Equal when the specified test is met, 7=NotEqual otherwise.

The structure and layout of the affected ACB are not part of the interface and are therefore not shown in this chapter. Direct access to subfields in the ACB is discouraged. Use SHOWCB ACB, TESTCB ACB and/or MODCB ACB to inspect, test, and/or modify the ACB's content.

Likewise, the structure and layout of the CBMR that zVSAM uses to transfer the TESTCB request to the CBMR handler are not part of the interface and are therefore not shown in this chapter. Direct access to subfields in the CBMR is strongly discouraged.

The TESTCB ACB macro can be coded as follows:

Opcode	Operand	Remarks
[label] TESTCB	ACB=address	Points TESTCB to the ACB to be tested
	ERET=addr	Address of error handling routine
	[OBJECT=DATA/INDEX]	For KSDS: select data or index component
	ATRB=(keywd_list) ATRB=COMPRESS ATRB=UNQ ATRB=XADDR	List of keywords indicating attributes to test Compression on? Always false for zVSAM. Path is defined on unique key? Extended format? Always true for zVSAM.
	OFLAGS=OPEN	Opened successfully?
	OPENOBJ=PATH/BASE/AIX	ACB represents Path/Base/AIX?
	ACBLEN=nr	length of ACB in bytes
	AVSPAC=nr	available space in bytes
	BSTRNO=nr	Initial nr of strings
	BUFND=nr	Nr of data buffers
	BUFNI=nr	Nr of index buffers
	BUFNO=nr	nr of I/O Buffers
	BUFSP=nr	Buffer space in bytes
	CINV=nr	Control interval size / Block size in bytes
	DDNAME=string	DDNAME
	ENDRBA=nr	High water mark XLRA
	ERROR=nr	Error code of last error
	EXLST=adr	EXLST address
	FS=nr	Free Block per 100
	KEYLEN=nr	Length of key field
	LRECL=nr	Logical Record Length
	MAREA=adr	Message area address

Opcode	Operand	Remarks
	MLen=nr	Length of message area in bytes
	NCIS=nr	Nr of Block splits
	NDELr=nr	Nr of deleted records
	NEXCP=nr	Nr of I/O requests
	NEXT=nr	Nr of extents
	NINSR=nr	Nr of records inserted
	NIXL=nr	Nr of index levels
	NLOGR=nr	Nr of records
	NRETR=nr	Nr of records retrieved
	NSSS=nr	Nr of control area splits. Foxes.
	NUPDR=nr	Nr of updates applied
	PASSWD=adr	Ptr to 1-byte length followed by password
	RKP=nr	Offset of key field within record
	SHRPOOL=nr	SHRPOOL number
	STMST=adr	Pojnter to system timestamp field
	STRNO=nr	Max. nr of parallel requests
	[MF=I or omitted]	Use standard form of TESTCB ACB
	[MF=L/MF=(L,addr,[label])]	Use list form of TESTCB ACB
	[MF=(E,addr)]	Use execute form of TESTCB ACB
	[MF=(G,addr,[label])]	Use generate form of TESTCB ACB

Supported options for the ATRB parameter are listed below:

Keyword	Remarks
ESDS	Component is an ESDS?
KSDS	Component is a KSDS?
LDS	Component is a LDS?
RRDS	Component is a RRDS?
REPL	Always false for zVSAM.
SPAN	Component may hold segmented records
SSWD	Always false for zVSAM.
VRRDS	Variable-length RRDS?
VESDS	Variable-length ESDS? (zVSAM extension)
WCK	Always false for zVSAM.

All supported parameters and keywords are implemented compatibly with IBM's VSAM implementation. For details, please refer to the relevant IBM manual.

For ease of access a short summary can be found in the addenda.

EXLST macro

The EXLST macro will generate an Exit_List control block and initialize it according to the parameters specified on the macro invocation.

The structure and layout of the generated EXLST are not part of the interface and are therefore not shown in this chapter. Direct access to subfields in the EXLST is discouraged. Use SHOWCB EXLST, TESTCB EXLST and/or MODCB EXLST to inspect, test, and/or modify the EXLST's content.

All keywords on the EXLST macro are optional. Before the cluster is opened, all EXLST values can be modified using MODCB EXLST, or by changing the EXLST directly. The latter is not recommended, as it is not guaranteed to be portable or compatible with future versions of zVSAM.

The table below shows how the EXLST macro can be coded.

Opcode	Operand	Remarks
[label] EXLST	[AM=VSAM]	Designates this EXLST as a zVSAM EXLST
	[EODAD=addr[,mod]]	End-of-data exit routine
	[LERAD=addr[,mod]]	Logical error analysis routine
	[SYNAD=addr[,mod]]	Physical error analysis routine
	[JRNAD=addr[,mod]]	Not supported – future option. Keyword is flagged as ignored with a warning message (Level 4 MNOTE)
	[UPAD=addr[,mod]]	Not supported – future option. Keyword is flagged as ignored with a warning message (Level 4 MNOTE)
	[RLSWAIT=addr[,mod]]	Not supported – future option. Keyword is flagged as ignored with a warning message (Level 4 MNOTE)

All supported parameters are implemented compatibly with IBM's VSAM implementation. For details, please refer to the relevant IBM manual.

For ease of access a short summary can be found in the addenda.

GENCB EXLST macro

The GENCB macro with EXLST=addr will generate or manipulate Exit Lists for use with ACBs and initialize or change them according to the parameters specified on the macro invocation. It is for this reason that all supported parameters and keywords of the EXLST macro (as described above) are supported on the GENCB macro when EXLST=exlst is specified.

The structure and layout of the generated EXLST are not part of the interface and are therefore not shown in this chapter. Direct access to subfields in the EXLST is discouraged. Use SHOWCB EXLST, TESTCB EXLST and/or MODCB EXLST to inspect, test, and/or modify the EXLST's content.

Likewise, the structure and layout of the CBMR that zVSAM uses to transfer the GENCB request to the CBMR handler are not part of the interface and are therefore not shown in this chapter. Direct access to subfields in the CBMR is strongly discouraged.

The GENCB EXLST macro can be coded as follows:

Opcode	Operand	Remarks
[label] GENCB	EXLST=addr	Instructs GENCB to generate 1 or more EXLSTs
	[AM=VSAM]	Optional, no other values allowed
	[COPIES=nr]	The number of identical EXLSTs to generate
	[WAREA=addr]	The work area where the EXLSTs are to be constructed
	[LENGTH=nr]	Length of the work area in bytes
	[LOC=keyword]	Where GENCB is to allocate dynamically acquired storage - if needed
	[other]	Any parameter supported on the EXLST macro
	[MF=I or omitted]	Use standard form of GENCB EXLST
	[MF=L/MF=(L, addr,[label])]	Use list form of GENCB EXLST
	[MF=(E,addr)]	Use execute form of GENCB EXLST
	[MF=(G,addr,[label])]	Use generate form of GENCB EXLST

All supported parameters are implemented compatibly with IBM's VSAM implementation. For details, please refer to the relevant IBM manual.

For ease of access a short summary can be found in the addenda.

MODCB EXLST macro

The MODCB macro with EXLST=addr will modify an EXLST according to the parameters specified on the macro invocation. It is for this reason that all parameters and keywords of the EXLST macro (as described above) are supported on the MODCB macro when EXLST=addr is specified.

The structure and layout of the affected EXLST are not part of the interface and are therefore not shown in this chapter. Direct access to subfields in the EXLST is discouraged. Use SHOWCB EXLST, TESTCB EXLST and/or MODCB EXLST to inspect, test, and/or modify the EXLST's content.

Likewise, the structure and layout of the CBMR that zVSAM uses to transfer the MODCB request to the CBMR handler are not part of the interface and are therefore not shown in this chapter. Direct access to subfields in the CBMR is strongly discouraged.

The MODCB EXLST macro can be coded as follows:

Opcode	Operand	Remarks
[label] MODCB	EXLST=addr	Points MODCB to the EXLST to be modified
	[other]	Any parameter supported on the EXLST macro
	[MF=I or omitted]	Use standard form of MODCB EXLST
	[MF=L/MF=(L,addr,[label])]	Use list form of MODCB EXLST
	[MF=(E,addr)]	Use execute form of MODCB EXLST
	[MF=(G,addr,[label])]	Use generate form of MODCB EXLST

All supported parameters are implemented compatibly with IBM's VSAM implementation. For details, please refer to the relevant IBM manual.

For ease of access a short summary can be found in the addenda.

SHOWCB EXLST macro

The SHOWCB macro with EXLST=addr will return ACB-related fields according to the parameters specified on the macro invocation in the order they are specified.

The structure and layout of the affected EXLST are not part of the interface and are therefore not shown in this chapter. Direct access to subfields in the EXLST is discouraged. Use SHOWCB EXLST, TESTCB EXLST and/or MODCB EXLST to inspect, test, and/or modify the EXLST's content.

Likewise, the structure and layout of the CBMR that zVSAM uses to transfer the SHOWCB request to the CBMR handler are not part of the interface and are therefore not shown in this chapter. Direct access to subfields in the CBMR is strongly discouraged.

The SHOWCB EXLST macro can be coded as follows:

Opcode	Operand	Remarks
[label] SHOWCB	EXLST=addr	Points MODCB to the EXLST to be queried
	AREA=addr	Address of return area
	LENGTH=nr	Size of return area in bytes
	[OBJECT=DATA/INDEX]	For KSDS: select data or index component
	FIELDS=(keywd_list)	List of keywords indicating which fields to return
	[MF=I or omitted]	Use standard form of SHOWCB EXLST
	[MF=L/MF=(L,addr,[label])]	Use list form of SHOWCB EXLST
	[MF=(E,addr)]	Use execute form of SHOWCB EXLST
	[MF=(G,addr,[label])]	Use generate form of SHOWCB EXLST

Supported options for the FIELDS parameter are listed below:

Keyword	Length	Remarks
EODAD	4	End-of-data exit routine address
EXLLEN	4	Size of EXLST in bytes
JRNAD	4	Not supported – future option. Keyword is flagged as ignored with a warning message (Level 4 MNOTE)
LERAD	4	Logical error analysis routine address
SYNAD	4	Physical error analysis routine address

All supported parameters and keywords are implemented compatibly with IBM's VSAM implementation. For details, please refer to the relevant IBM manual.

For ease of access a short summary can be found in the addenda.

TESTCB EXLST macro

The TESTCB macro with EXLST=addr will test EXLST-related fields according to the parameters specified on the macro invocation. Only a single test can be specified on each TESTCB invocation. TESTCB returns a PSW condition code.

The structure and layout of the affected EXLST are not part of the interface and are therefore not shown in this chapter. Direct access to subfields in the EXLST is discouraged. Use SHOWCB EXLST, TESTCB EXLST and/or MODCB EXLST to inspect, test, and/or modify the EXLST's content.

Likewise, the structure and layout of the CBMR that zVSAM uses to transfer the TESTCB request to the CBMR handler are not part of the interface and are therefore not shown in this chapter. Direct access to subfields in the CBMR is strongly discouraged.

The TESTCB EXLST macro can be coded as follows:

Opcode	Operand	Remarks
[label] TESTCB	EXLST=addr	Points TESTCB to the EXLST to be tested
	ERET=addr	Address of error handling routine
	EODAD=0 EODAD=addr[,mod]	End-of-data exit routine address
	JRNAD=0 JRMAD=addr[,mod]	Not supported – future option. Keyword is flagged as ignored with a warning message (Level 4 MNOTE)
	LREAD=0 LERAD=addr[,mod]	Logical error analysis routine address
	SYNAD=0 SYNAD=addr[,mod]	Physical error analysis routine address
	EXLLEN=nr	Size of EXLST in bytes
	[MF=I or omitted]	Use standard form of TESTCB EXLST
	[MF=L/MF=(L,addr,[label])]	Use list form of TESTCB EXLST
	[MF=(E,addr)]	Use execute form of TESTCB EXLST
	[MF=(G,addr,[label])]	Use generate form of TESTCB EXLST

OPEN macro

A cluster needs to be opened before it can be processed. The open macro is used to open one or more clusters and/or one or more sequential files in a single call.

Opcode	Operand	Remarks
[label] OPEN	(entry[,entry]...)	Each cluster or file requires an entry of two parameters
	[MODE= <u>24</u> /31]	Residency mode of all control blocks involved. Specify 31 if any resides above the line
Entry format:	addr,(options)	Address of ACB or DCB, followed by a list of options (for DCB only). For ACB omit the list of options.
	[MF=I or omitted]	Use standard form of OPEN
	[MF=L/MF=(L,addr)]	Use list form of OPEN
	[MF=(E,addr)]	Use execute form of OPEN

All supported parameters are implemented compatibly with IBM's VSAM implementation. For details, please refer to the relevant IBM manual.

For ease of access a short summary can be found in the addenda.

CLOSE macro

A cluster needs to be closed after it has been processed. The close macro is used to close one or more clusters and/or one or more sequential files in a single call.

Opcode	Operand	Remarks
[label] CLOSE	(entry[,entry]...)	Each cluster or file requires an entry of two parameters
	[MODE= <u>24</u> /31]	Residency mode of all control blocks involved. Specify 31 if any resides above the line
	[TYPE=T]	Not supported – future option. Keyword is flagged as ignored with a warning message (Level 4 MNOTE)
Entry format:	addr,(options)	Address of ACB or DCB, followed by a list of options (for DCB only). For ACB omit the list of options.
	[MF=I or omitted]	Use standard form of CLOSE
	[MF=L/MF=(L,addr)]	Use list form of CLOSE
	[MF=(E,addr)]	Use execute form of CLOSE

All supported parameters are implemented compatibly with IBM's VSAM implementation. For details, please refer to the relevant IBM manual.

For ease of access a short summary can be found in the addenda.

RPL-based interfaces

The RPL is the primary interface for operations at the record level:

A program can use multiple RPLs. An RPL must always point to an open ACB in order to specify a valid operation.

- RPL.Get
- RPL.Put
- RPL.Erase
- RPL.Point
- RPL.EndReq
- Support for other RPL-based requests is reserved for a future release

RPL macro

The RPL macro will generate an RPL and initialize it according to the parameters specified on the macro invocation.

The structure and layout of the generated RPL are not part of the interface and are therefore not shown in this chapter. Direct access to subfields in the RPL is discouraged. Use SHOWCB RPL, TESTCB RPL and/or MODCB RPL to inspect, test, and/or modify the RPL's content.

All keywords on the RPL macro are optional. Before a request is issued, all RPL values can be modified using MODCB RPL, or by changing the RPL directly. The latter is not recommended, as it is not guaranteed to be portable or compatible with future versions of zVSAM.

The table below shows how the RPL macro can be coded.

Opcode	Operand	Remarks
[label] RPL	[AM=VSAM]	Designates this RPL as a zVSAM RPL
	[ACB=ptr]	Pointer to ACB
	[AREA=ptr]	Pointer to record area or record pointer
	[AREALEN=nr]	Length of record area or record pointer
	[ARG=ptr]	Pointer to search argument
	[ECB=]	Pointer to ECB
	[KEYLEN=nr]	Length of search argument
	[MSGAREA=addr]	Pointer to message area
	[MSGLEN=nr]	Length of message area
	[NXTRPL=ptr]	Pointer to next RPL when chaining requests
	[OPTCD=(keywd_list)]	List of keywords specifying processing options. See table below for valid keywords
	[RECLen=nr]	Max amount of storage (in bytes) to use for buffers
	[TRANSID=nr]	Not supported – future option. Keyword is flagged as ignored with a warning message (Level 4 MNOTE)

Supported options for the OPTCD parameter are listed below:

Keyword subset	Keyword	Remarks
[ADR <u>KEY</u>]	ADR	Addressed access to ESDS or KSDS(under review)
	KEY	Keyed access to KSDS or RRDS
	CNV	Not supported – future option. Keyword is flagged as ignored with a warning message (Level 4 MNOTE)
[DIR <u>SEQ</u> SKP]	DIR	Direct access to ESDS, KSDS, RRDS
	SEQ	Sequential access to ESDS, KSDS or RRDS
	SKP	Skip sequential access to KSDS or RRDS
[<u>ARD</u> LRD]	ARD	Access user-defined record location
	LRD	Access last record in the cluster
[<u>FWD</u> BWD]	FWD	Forward processing
	BWD	Backward processing
[<u>SYN</u> ASY]	SYN	Synchronous request
	ASY	Asynchronous request
[<u>NUP</u> UPD NSP]	NUP	Not for update
	UPD	For update
	NSP	Retain positioning for next sequential access
[<u>KEQ</u> KGE]	KEQ	Locate record with exact key match
	KGE	Locate record with exact key match, or next higher value
[<u>FKS</u> GEN]	FKS	Full key search
	GEN	Generic key search. KEYLEN required
[<u>MVE</u> LOC]	MVE	Move mode
	LOC	Locate mode
[<u>RBA</u> XRBA]	RBA	4-byte RBA values
	XRBA	8-byte extended RBA values
[NWAITX/WAITX]		Not supported – future option. Keyword is flagged as ignored with a warning message (Level 4 MNOTE)
[CR/NRI]		Not supported – future option. Keyword is flagged as ignored with a warning message (Level 4 MNOTE)

All supported parameters are implemented compatibly with IBM's VSAM implementation. For details, please refer to the relevant IBM manual

For ease of access a short summary can be found in the addenda

GENCB RPL macro

The GENCB macro with RPL=addr will generate or manipulate RPLs and initialize or change them according to the parameters specified on the macro invocation. It is for this reason that all supported parameters and keywords of the RPL macro (as described above) are supported on the GENCB macro when RPL=addr is specified.

The structure and layout of the generated RPL are not part of the interface and are therefore not shown in this chapter. Direct access to subfields in the RPL is discouraged. Use SHOWCB RPL, TESTCB RPL and/or MODCB RPL to inspect, test, and/or modify the RPL's content.

Likewise, the structure and layout of the CBMR that zVSAM uses to transfer the GENCB request to the CBMR handler are not part of the interface and are therefore not shown in this chapter. Direct access to subfields in the CBMR is strongly discouraged.

The GENCB RPL macro can be coded as follows:

Opcode	Operand	Remarks
[label] GENCB	RPL=addr	Instructs GENCB to generate 1 or more RPLs
	[AM=VSAM]	Optional, no other values allowed
	[COPIES=nr]	The number of identical RPLs to generate
	[WAREA=addr]	The work area where the RPLs are to be constructed
	[LENGTH=nr]	Length of the work area in bytes
	[LOC=keyword]	Where GENCB is to allocate dynamically acquired storage - if needed
	[other]	Any parameter supported on the RPL macro
	[MF=I or omitted]	Use standard form of GENCB RPL
	[MF=L/MF=(L,addr,[label])]	Use list form of GENCB RPL
	[MF=(E,addr)]	Use execute form of GENCB RPL
	[MF=(G,addr,[label])]	Use generate form of GENCB RPL

All supported parameters are implemented compatibly with IBM's VSAM implementation. For details, please refer to the relevant IBM manual

For ease of access a short summary can be found in the addenda

MODCB RPL macro

The MODCB macro with RPL=addr will modify an RPL according to the parameters specified on the macro invocation. It is for this reason that all parameters and keywords of the RPL macro (as described above) are supported on the MODCB macro when RPL=addr is specified.

The structure and layout of the affected RPL are not part of the interface and are therefore not shown in this chapter. Direct access to subfields in the RPL is discouraged. Use SHOWCB RPL, TESTCB RPL and/or MODCB RPL to inspect, test, and/or modify the RPL's content.

Likewise, the structure and layout of the CBMR that zVSAM uses to transfer the MODCB request to the CBMR handler are not part of the interface and are therefore not shown in this chapter. Direct access to subfields in the CBMR is strongly discouraged.

The MODCB RPL macro can be coded as follows:

Opcode	Operand	Remarks
[label] MODCB	RPL=addr	Points MODCB to the RPL to be modified
	[other]	Any parameter supported on the RPL macro
	[MF=I or omitted]	Use standard form of MODCB RPL
	[MF=L/MF=(L,addr,[label])]	Use list form of MODCB RPL
	[MF=(E,addr)]	Use execute form of MODCB RPL
	[MF=(G,addr,[label])]	Use generate form of MODCB RPL

All supported parameters are implemented compatibly with IBM's VSAM implementation. For details, please refer to the relevant IBM manual.

For ease of access a short summary can be found in the addenda.

SHOWCB RPL macro

The SHOWCB macro with RPL=addr will return RPL-related fields according to the parameters specified on the macro invocation in the order they are specified.

The structure and layout of the affected RPL are not part of the interface and are therefore not shown in this chapter. Direct access to subfields in the RPL is discouraged. Use SHOWCB RPL, TESTCB RPL and/or MODCB RPL to inspect, test, and/or modify the RPL's content.

Likewise, the structure and layout of the CBMR that zVSAM uses to transfer the SHOWCB request to the CBMR handler are not part of the interface and are therefore not shown in this chapter. Direct access to subfields in the CBMR is strongly discouraged.

The SHOWCB RPL macro can be coded as follows:

Opcode	Operand	Remarks
[label] SHOWCB	RPL=addr	Points SHOWCB to the RPL to be queried
	AREA=addr	Address of return area
	LENGTH=nr	Size of return area in bytes
	FIELDS=(keywd_list)	List of keywords indicating which fields to return
	[MF=I or omitted]	Use standard form of SHOWCB RPL
	[MF=L/MF=(L,addr,[label])]	Use list form of SHOWCB RPL
	[MF=(E,addr)]	Use execute form of SHOWCB RPL
	[MF=(G,addr,[label])]	Use generate form of SHOWCB RPL

Supported options for the FIELDS parameter are listed below:

Keyword	Length	Remarks
ACB	4	Pointer to ACB
AIXPC	4	Alternate index pointer count
AREA	4	Pointer to record buffer
AREALEN	4	Size of record buffer in bytes
ARG	4	Pointer to last used search argument field
ECB	4	Pointer to user-supplied ECB
FDBK	4	Feedback code for the last request
FTNCD	4	Function code
KEYLEN	4	Length of key, for use with OPTCD=GEN
MSGAREA	4	Pointer to message area, foxes if not relevant
MSGLEN	4	Length of message area, foxes if not relevant
NXTRPL	4	Pointer to next RPL, if any
RBA	4	4-byte RBA of last record processed
RECLen	4	Length of current record

Keyword	Length	Remarks
RPLLEN	4	Length of RPL
TRANSID	4	Transaction_id. Always foxes.
XRBA	8	8-byte RBA of last record processed

All supported parameters and keywords are implemented compatibly with IBM's VSAM implementation. For details, please refer to the relevant IBM manual.

For ease of access a short summary can be found in the addenda.

Overview of differences with IBM VSAM:

FIELDS=RBA/XRBA – zVSAM supports these keywords only for ESDS. For any other type of cluster a value of foxes will be returned by default

TESTCB RPL macro

The TESTCB macro with RPL=addr will test RPL-related fields according to the parameters specified on the macro invocation. Only a single test can be specified on each TESTCB invocation. TESTCB returns a PSW condition code of 8=Equal when the specified test is met, 7=NotEqual otherwise.

The structure and layout of the affected RPL are not part of the interface and are therefore not shown in this chapter. Direct access to subfields in the RPL is discouraged. Use SHOWCB RPL, TESTCB RPL and/or MODCB RPL to inspect, test, and/or modify the RPL's content.

Likewise, the structure and layout of the CBMR that zVSAM uses to transfer the TESTCB request to the CBMR handler are not part of the interface and are therefore not shown in this chapter. Direct access to subfields in the CBMR is strongly discouraged.

The TESTCB RPL macro can be coded as follows:

Opcode	Operand	Remarks
[label] TESTCB	RPL=addr	Points TESTCB to the RPL to be tested
	ERET=addr	Address of error handling routine
	OPTCD=(keywd_list)	List of keywords indicating attributes to test
	AIXFLAG=AIXPKP	Using primary keys
	AIXPC=nr	Nr of index pointers in use
	FTNCD=nr	Reflects the condition of the upgrade set
	IO=COMPLETE	
	ACB=addr	
	AREA=addr	
	AREALEN=addr	
	ARG=addr	
	ECB=addr	
	FDBK=nr	
	KEYLEN=nr	Length of key field
	RECLen=nr	Logical Record Length
	MSGAREA=adr	Message area address
	MSGLEN=nr	Length of message area in bytes
	NXTRPL=addr	
	RBA=nr	
	RPLEN=nr	
	TRANSID=nr	
	[MF=I or omitted]	Use standard form of TESTCB ACB
	[MF=L/MF=(L,addr,[label])]	Use list form of TESTCB ACB
	[MF=(E,addr)]	Use execute form of TESTCB ACB

Opcode	Operand	Remarks
	[MF=(G,addr,[label])]	Use generate form of TESTCB ACB

Supported options for the OPTCD parameter are the same as those available on the RPL macro.

All supported parameters and keywords are implemented compatibly with IBM's VSAM implementation. For details, please refer to the relevant IBM manual.

For ease of access a short summary can be found in the addenda.

Overview of differences with IBM VSAM:

RBA=nr – zVSAM supports this keyword only for ESDS. For any other type of cluster a value of foxes will be assumed by default.

POINT macro

GET macro

PUT macro

ERASE macro

CHECK macro

ENDREQ macro

VERIFY macro

Catalog management

This is where all meta-data about the zVSAM components are kept and where the relations between zVSAM components are defined. Catalogs are currently created as static assembled modules. Extended catalogs contained in datasets will be considered in a future release.

The catalog will hold at least:

- file name
- pointer to index file
- pointers to all related AIX clusters
- LRECL
- record type (F, V, FS, VS)
- type of component (ESDS, KSDS, RRDS, LDS, AIX)
- freeblocks (during load, between blocks)
- freespace (during load, within blocks)
- Physical Block size (aka CI-size, 512 bytes to 16MB)

Physical structure of the files

Basic Concepts

Files, Blocks, Records

The logical unit of access or storage is the record. Yet the unit for any given I/O operation is the block. Block sizes may vary from 512 bytes to 16MB. Each block holds up to 255 records. For any given cluster component, choosing an appropriate block size is important. Block size can greatly affect not only performance, but also both internal and external storage consumption.

A cluster consists of one or more files that belong together and should be managed together. Whether you take a backup, perform a restore, or perform other administrative tasks, the files that make up a cluster should be managed alike. When creating a backup copy of a cluster or restoring a cluster, make sure no other processes try to access the data at the same time.

zVSAM implements a number of checks and balances to prevent inadvertent access to data that may have been compromised. Names and locations of files are managed. Tampering with files or file attributes may render the cluster unusable.

As a result, it is not possible to rename a zVSAM cluster or file. Unload and reload your cluster in order to move the data or to assign a different name to cluster or file.

Just like files in a cluster belong together and should be managed together, clusters in a sphere are logically connected and should be managed together. Again, failing to manage the files in a correct and comprehensive manner may render your data inaccessible.

Cluster types and Cluster Components

Each cluster consists of a data component and an index component as follows:

Data and Index Components per Cluster type	
Cluster type	Index content
ESDS	Index on RBA
KSDS	Index on key value or RBA (under review)
RRDS	Index on RRN
LDS	No index
AIX	Index on alternate key value

Record Formats

In zVSAM we recognize and support the following record formats:

Record Formats	
Format	Properties
F	Fixed. All records have the same length. Records never span a Block boundary.
FS	Fixed Spanned. All records have the same length. Records expected to span a Block boundary.
V	Variable. Records have varying lengths. Records never span a Block boundary.
VS	Variable Spanned. Records have varying lengths. Records may or may not span a Block boundary.

For ESDS, KSDS, and RRDS all record types are supported. For LDS All records are 4096 bytes. For AIX only F and VS record formats are supported: F for unique, and VS for non-unique indexes

Supported Record Formats per Cluster Type				
Cluster Type	F	FS	V	VS
ESDS	Y	Y	Y*	Y*
KSDS	Y	Y	Y	Y
RRDS	Y	Y*	Y	Y*
LDS	N	N	N	N
AIX - unique	Y	N	N	N
AIX - non-unique	N	N	N	Y
* zVSAM extension				

For a unique AIX each record holds an alternate key value plus the primary key (KSDS) or XLRA (ESDS) of the associated record in the cluster's data component. This fixed configuration dictates a record type of F.

For a non-unique AIX each record holds an alternate key value and as many primary keys (KSDS) or XLRA (ESDS) of associated records in the cluster's data component as there are records holding that specific alternate key value. The table of primary keys may vary in length from 1 to very large numbers. No block size is guaranteed to be large enough to hold the largest possible index record, therefore a record type of VS is mandated. When a non-unique index record needs to be split into segments, no primary key value or XLRA is ever split; i.e. only an exact number of these reside within a single segment of the record .

Supported Index-types per Cluster Type			
Cluster Type	Primary - Unique	AIX _unique	AIX - Non-unique
ESDS	Y*	Y	Y
KSDS	Y	Y	Y
RRDS	Y*	N	N
LDS	N	N	N
* zVSAM extension			

Concept of Fixed length records stored in blocks

Disregarding block structure elements, F-type records are conceptually stored one after another, filling the block until no space is left. When remaining free space is insufficient to accommodate another record, that free space remains unallocated (marked in blue). The actual implementation is quite different, but we'll leave those details alone for the moment.

Below we show an example where two records will fit into each block:

Block 1	Record 1	Record 2	Unallocated
Block 2	Record 3	Record 4	Unallocated
Etc.	Many more blocks like the above ones		
Last Block	Last record	Unallocated	

This holds for all cluster types, except LDS. In an LDS there is no block structure. Each block and each record holds 4096 bytes of data.

Below we show an example of records in a LDS:

Block 1	Record 1 (4096 bytes)
Block 2	Record 2 (4096 bytes)
Etc.	Many more blocks like the above ones
Last Block	Last record (4096 bytes)

Concept of Fixed length Segmented records stored in blocks

Disregarding block structure elements, FS-type records are conceptually stored one after another, using a block for each segment and starting each record on a new block. Record size is expected to exceed block size, so the record is split into segments, the first segment is created to fill an entire block, and the rest of the record goes into a second segment, which is stored on the next block. Each segment is preceded by a Segment Prefix (SPX, marked in yellow). Depending on record size and usable block size, more than two segments may be needed to store the record. The actual implementation is quite different, but we'll leave those details alone for the moment.

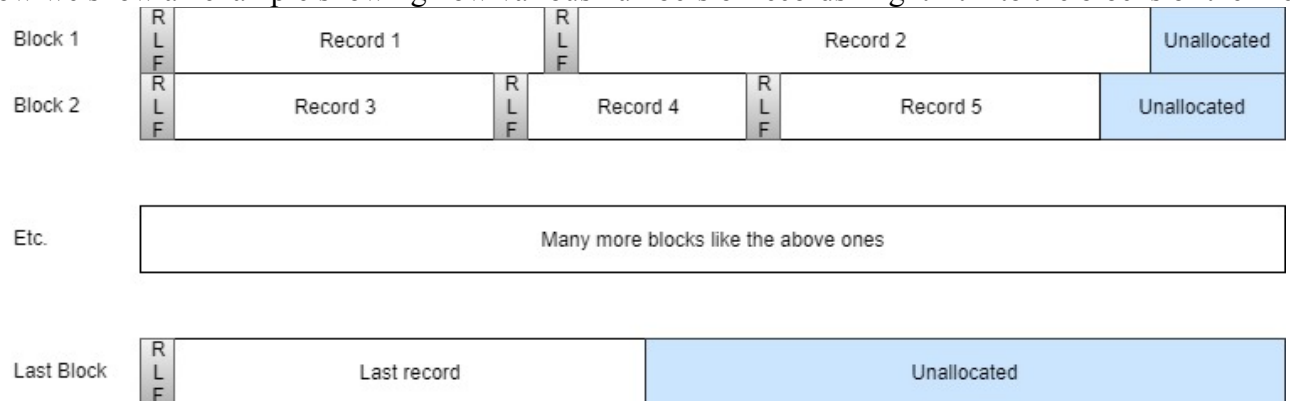
Below we show an example where each record requires three blocks and is therefore split into three segments:

Block 1	S P X	Record 1 - Segment 1	
Block 2	S P X	Record 1 - Segment 2	
Block 3	S P X	Record 1 - Segment 3	Unallocated
Block 4	S P X	Record 2 - Segment 1	
Block 5	S P X	Record 2 - Segment 2	
Block 6	S P X	Record 2 - Segment 3	Unallocated
Etc.	Many more sets of blocks like the above ones		
Last but 2	S P X	Last Record - Segment 1	
Last but 1	S P X	Last Record - Segment 2	
Last Block	S P X	Last Record - Segment 3	Unallocated

Concept of Variable length records stored in blocks

Disregarding block structure elements, V-type records are conceptually stored one after another, filling the block until no space is left. When remaining free space is insufficient to accommodate another record, that free space remains unallocated (marked in blue). Every record is preceded by a Record Length Field (RLF, marked in grey). The actual implementation is quite different, but we'll leave those details alone for the moment.

Below we show an example showing how various numbers of records might fit into the blocks of the file:



Concept of Variable length Segmented records stored in blocks

Disregarding block structure elements, VS-type records are conceptually stored one after another, filling the block until no space is left. Every record is preceded by a Record Length Field (RLF, marked in grey). When remaining free space is insufficient to accommodate a complete record, the record is placed on the next block. Only if the record size exceeds usable block size, the record is split into segments and each segment is prefixed with a Segment Prefix. The first segment is created to fill a block and the rest of the record goes into a second segment, which is stored on the next block. Each segment is preceded by a Segment Prefix (SPX, marked in yellow). Please note that the RLF occurs only once in each record, whereas each record segment has its own SPX. Depending on record size and usable block size, more than two segments may be needed to store a record. The actual implementation is quite different, but we'll leave those details alone for the moment.

Below we show an example showing how various numbers of records might fit into the blocks of the file, or how a single record might occupy multiple blocks of the file

Block 1	R L F	Record 1	R L F	Record 2	Unallocated
Block 2	R L F	Record 3			Unallocated
Block 3	S P X	R L F	Record 4 - Segment 1		
Block 4	S P X	Record 4 - Segment 2		Unallocated	
Block 5	S P X	R L F	Record 5 - Segment 1		
Block 6	S P X	Record 5 - Segment 2			
Block 7	S P X	Record 5 - Segment 3			Unallocated
Etc.	Many more blocks like the above ones				
Last Block	R L F	Last record		Unallocated	

File Structure

Physical files

All zVSAM data is stored in physical files, as defined to the operating system. Each component consists of one file. This file is formatted as a zVSAM file. The structure of which is explained in the next set of chapters.

Please note: the hosting operating system may impose a limit on physical file size and not every host OS supports a physical file spanning a volume boundary of the storage device(s). Therefore, to support clusters that exceed the maximum size of a single physical file, in the future we may need to support clusters that consist of multiple files.

Structure of physical files

Every zVSAM file has a block size. The block being the basic unit of I/O. The first block of every file is the prefix block, which is always 4096 bytes in size. The prefix block holds information about the cluster, its data, and its structure.

Data in the prefix block are not accessible to user programs. However, selected fields in the prefix block can be queried using a SHOWCB ACB request.

All other blocks in the file have a user-defined blocksize. That is, the user defines the blocksize. All blocks in the file are created with that size, except the prefix block which is always 4096 bytes, irrespective of the size of the other blocks in the file. The file is assumed to logically begin with the first block after the prefix block.

There are 6 types of blocks that may occur in zVSAM files:

- 1) Prefix block – one for each file, being the first 4096 bytes of every file
- 2) Spacemap block – used to manage free space in the file
- 3) Data block – used to hold user data, or AIX data records (in an AIX only)
- 4) Index block – used to hold index information
- 5) ELIX block – used to index segmented (read: large) non-unique AIX records
- 6) Raw block – used to hold a block's worth of LDS data

Every block, except a raw block, has an internal structure, consisting of a block header, a list of record pointers (data and index blocks only), a block body and a block footer. The block header and footer have a fixed structure. The list of record pointers, if present, has a variable length. The block body contains record data and/or free space.

Raw blocks have no internal structure, as far as zVSAM is concerned. Any and all internal structure(s) in an LDS are to be maintained by the application program.

Each of the 5 block types is explained in more detail below.

Prefix Block

The prefix block consists of the first 4096 bytes of every physical file. It contains meta-data defining the file and its attributes. It also contains various counters.

The prefix block consists of a block header immediately followed by the prefix area described below. The prefix block also contains other data fields, these are addressed from the prefix area. The prefix block ends with a block footer. A record pointer list is not present on the prefix block.

There are various pointer fields in the prefix area. These point to fields allocated elsewhere in the prefix block. Their exact addresses on the prefix block may vary.

The PFXDVOL@, PFXDPAT@, PFXDNAM@ pointers and the PFXXVOL@, PFXXPAT@, PFXXNAM@ all point to a halfword-prefixed string.

The PFXCTRS@ pointer addresses a separate area that holds various counters. This area is expected to move into the catalog dataset in a future release.

The overall structure of the prefix block would look something like this (areas not to scale):

Block Header	Prefix area - part 1	
Prefix area - part 2		
Counters area	Free space	
Free space		
Free space	Other fields addressed from Prefix area	Block Footer

Spacemap Blocks

Spacemap blocks are used to manage available free space in a component. Each spacemap block has a size that matches the blocksize of all other blocks (except possibly the prefix block) in the component.

A component will hold as many spacemap blocks as needed to map all of its allocated blocks, including all spacemap blocks but excluding the prefix block. Whenever a single spacemap block is not enough, the spacemap blocks are chained together by means of the BHDRNEXT/BHDRPREV pointers in the block header area. The spacemap chain starts/ends from the prefix block, fields PFXBMAP/PFXEMAP.

When a single spacemap block suffices, PFXBMAP and PFXEMAP will both point to that block.

Each spacemap block consists of a block header immediately followed by the spacemap area, which in turn is followed directly by the block footer. No free space exists on a spacemap block. Thus, a spacemap block may indicate blocks that do not exist in the dataset. The bit settings for blocks beyond the PFXHXLRA should all be zero to indicate an unallocated block. zVSAM is aware that any block beyond PFXHXLRA needs to be created and initialized before it can be allocated.

Conceptually, the overall structure of a spacemap block would look something like this (areas not to scale):

Block Header	Spacemap area	
Spacemap area		
Spacemap area		
Spacemap area		Block Footer

Data Blocks

Index Blocks

Raw Blocks

Structure of ESDS

Structure of KSDS

Structure of RRDS

Structure of LDS

Free Blocks

Block Structures

Block Header Structure

Every block (except raw blocks) has a block header. All block headers have the same structure. It is formatted as follows:

Label	Offset	Field type	Function
ZVSAMHDR		DSECT	Block header area
BHDREYE	X'000'	CL3	=C'HDR' – eyecatcher to mark the area
BHDRSEQ#	X'003'	XL1	Write control value
BHDRVER	X'004'	XL1	Design sequence number
BHDR_V2		=X'02'	Current design version number
BHDRFLGS	X'005'	XL1	Flags
BHDR_PFX		=X'80'	Prefix block
BHDR_MAP		=X'40'	Spacemap block
BHDR_DTA		=X'20'	Data block
BHDR_IDX		=X'10'	Index block
BHDR_SEG		=X'08'	Segment block
BHDR_LEF		=X'04'	Index leaf Block
BHDR_INT		=X'02'	Index intermediate block
BHDR_ROT		=X'01'	Index root block
BHDR#REC	X'006'	XL1	Nr of records on this block
BHDRXLVL	X'007'	XL1	Index level
BHDRSELF	X'008'	XL8	XLRA of this block
BHDRNEXT	X'010'	XL8	XLRA of next block on chain
BHDRPREV	X'018'	XL8	XLRA of previous block on chain
BHDRFRE@	X'020'	XL3	Offset of free area on this block
BHDRFREE	X'024'	XL3	Length of free area on this block
		XL2	Reserved

BHDRSEQ# is incremented by one every time the block is written out to the file. The footer area contains a comparable field: BFTRSEQ#. Together they guard against incomplete writes.

BHDRXLVL indicates the index level. Zero is the leaf level. Index blocks are chained by level. That is, for every index level in use there is a pair of pointers in the prefix block (PFXBLVLn/PFXELVLn) that starts and ends the chain for that level.

BHDRSELF contains the block's own XLRA. This helps to guard against misdirected reads and/or writes.

BHDRNEXT/BHDRPREV point to the next and previous block on the chain. Which chain this is, depends on the BHDRFLAG setting, and, if this is an index block, by the BHDRXLVL value. For the prefix block, these two fields are set to foxes.

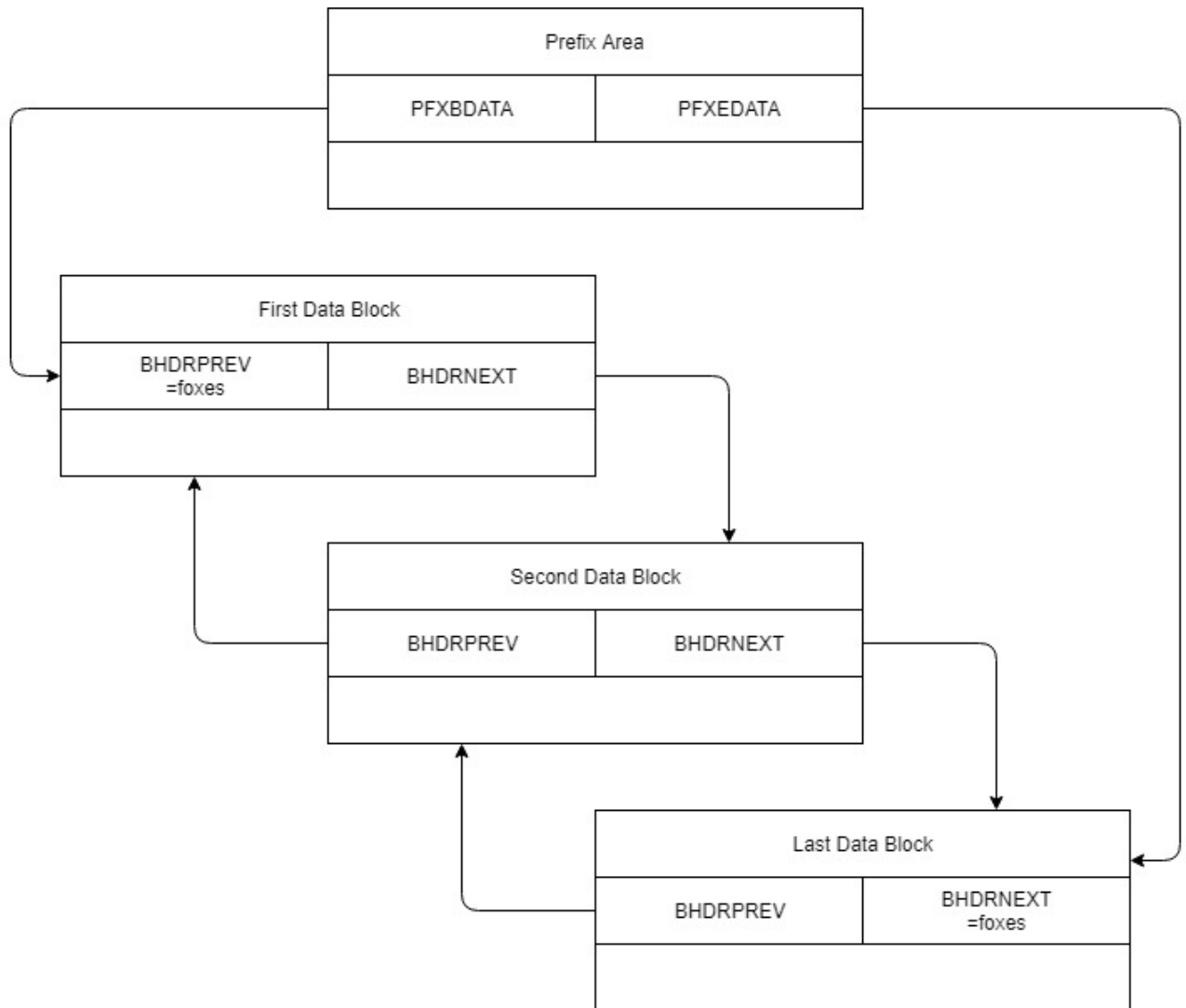
Segmented records are a special case. Segments of a segmented record never share their block with other data. The block holding the first segment is part of the data chain. A block holding a non-first segment is part of the segment chain. A block that holds a record's first segment has an SPX pointing into the segment chain at the block holding the second segment. From the second segment the BHDRPREV/NEXT chain can be used to sequentially read subsequent segments up to the last segment of the record.

The following table summarizes the way that blocks in the file are chained from the prefix block.

The prefix block doesn't reside on any chain

Block Type	Begin of chain	End of chain
Prefix	foxes	foxes
Spacemap	PFXBMAP	PFXEMAP
Data (in use and free)	PFXBDATA	PFXEDATA
Data (non-first segment)	PFXBSEGM	PFXESEGM
Index (in use and free)	PFXBLVLn	PFXELVLn

Example 1: Assume we have a cluster with three data blocks holding unsegmented records. The blocks are on the data chain as outlined in the picture below. Please note that all depicted pointers are block pointers. Each pointer thus originates with the indicated field, and ends at the block it points to. The location where the arrows attach has no meaning since it's a block pointer.

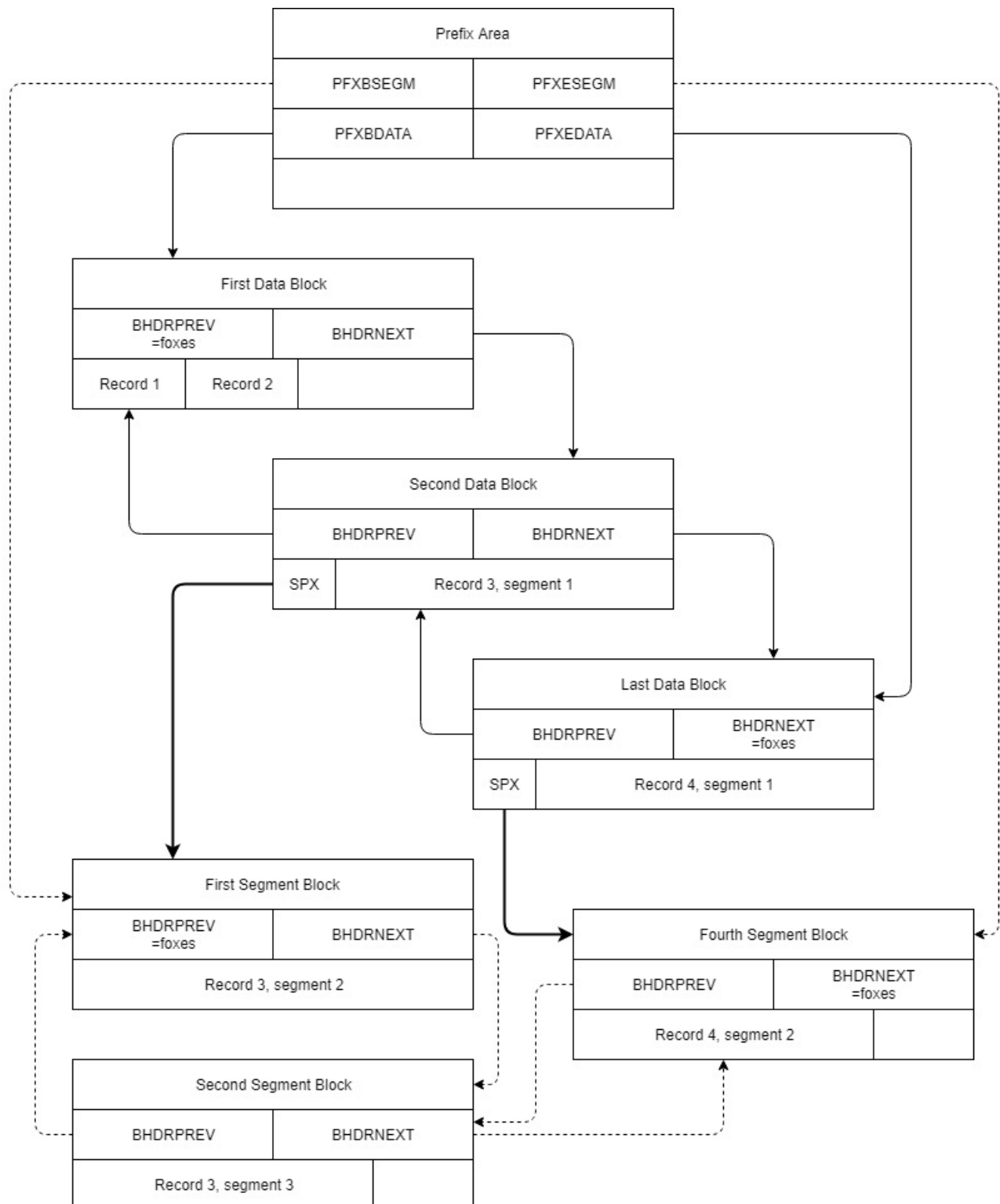


Example 2: Now suppose we have a cluster with three data blocks, the first block holding two unsegmented records, the second block holding the first segment of a record consisting of three segments and the third block holding the first segment of a record consisting of two segments.

In the picture (see next page) we show the data chain as a solid line (as in the picture above), we show the segment chain as a dotted line, and we show the SPX pointers as a fat line.

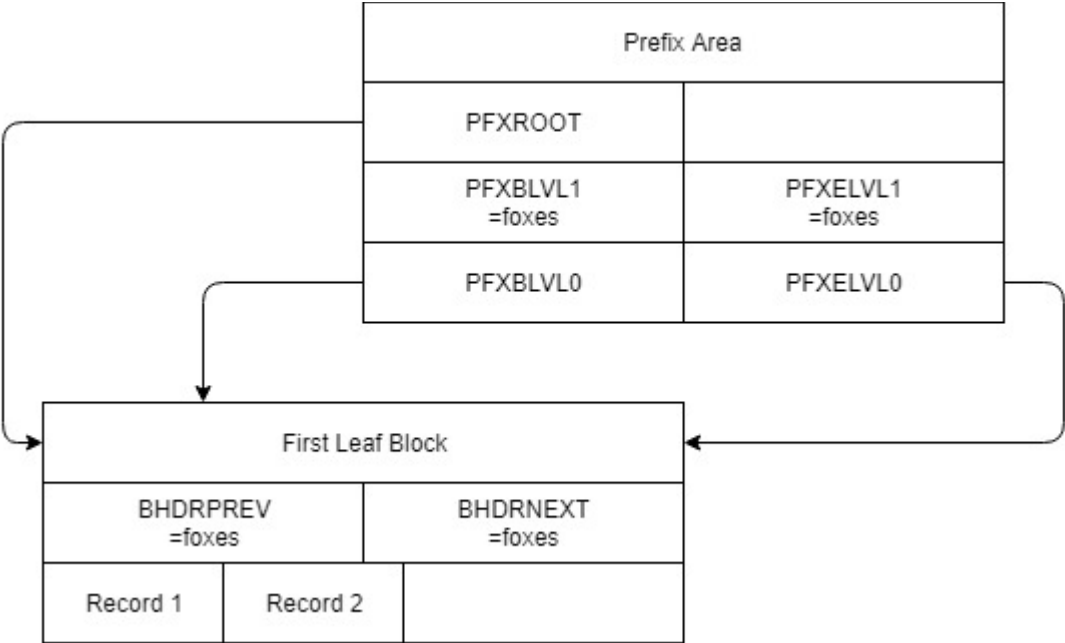
The picture shows the prefix area's pointer to start/end block of both the data chain and the segment chain. It also shows the first and second block on each chain pointing to one another. Same thing for the second and third block on each chain.

###Diagram incorrect, SPX occurs on each segment

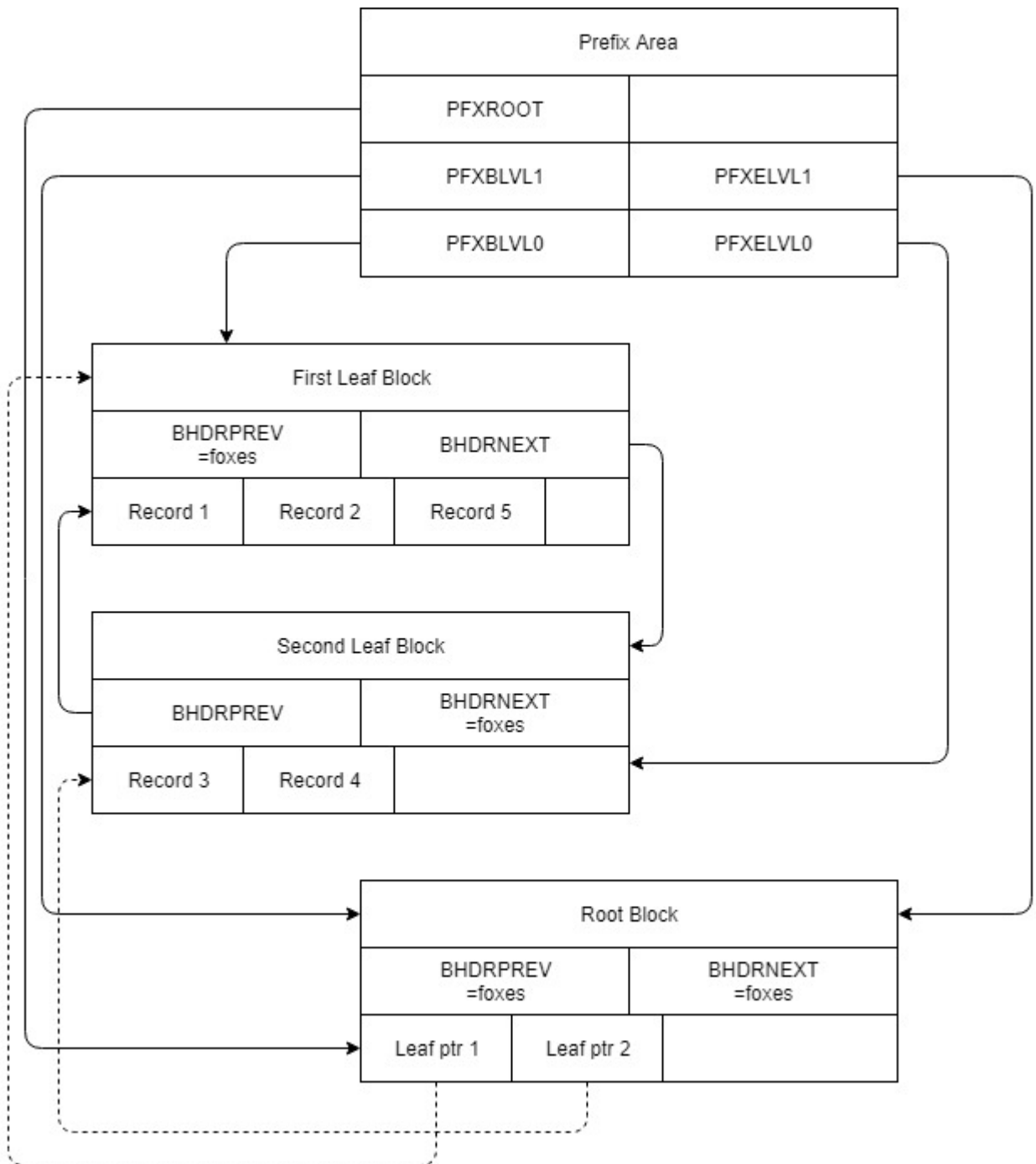


As in the example above, all depicted pointers are block pointers. Each pointer originates with the indicated field, and ends at the block it points to. The location where the arrows attach has no meaning since it's a block pointer.

Example 3: This example shows an index of only one block, holding two record pointers.



Example 4: This example shows the index after adding three more record pointers, causing the only index block to overflow and split. Now there are two leaf blocks, still on the LVL0 chain, and a new root block has been created on the LVL1 chain.



Block Footer Structure

Every block (except raw blocks) has a block footer. All block footers have the same structure. It is formatted as follows:

Label	Offset	Field type	Function
ZVSAMFTR		DSECT	Block header area
BFTREYE	X'000'	CL3	=C'FTR' – eyecatcher to mark the area
BFTRSEQ#	X'003'	XL1	Write control value

BFTRSEQ# is incremented by one every time the block is written out to the file. The header area contains a comparable field: BHDRSEQ#. Together they guard against incomplete writes.

Record Pointer List Structure

Every block that contains data records contains a record pointer list. Records are accessible only through their Record Pointer or RPTR. Every entry in the list corresponds with a single record on the block. The last byte of record's XLRA is the index into the Record Pointer List. Index value of X'00' is reserved for block pointers; values X'01' through X'FF' inclusive are usable as RPTR index values. The difference of 1 always needs to be taken into account when indexing the RPTR list.

The RPTR list always follows the block header directly.

The number of entries on the RPTR list varies with the number of records stored on the block (BHDR#REC) and is terminated with an entry of foxes to mark the end of the list.

Record Pointer List entries are formatted as follows:

Label	Offset	Field type	Function
ZVSAMRPT		DSECT	Record Pointer
RPTRFLGS	X'000'	XL1	Flag byte
RPTR_ACT		X'80'	Active record
RPTR_MTY		X'40'	Empty record slot
RPTR_DIS		X'20'	Record has been displaced to another block
RPTR_MOV		X'10'	New location of a moved record
RPTR_SEG		X'08'	Record segment
RPTR_END		X'01'	Terminating entry
RPTRREC@	X'001'	AL3	Record offset within block

When RPTR_END is set, all bits in the RPTR entry should be on.

RPTR_ACT and RPTR_MTY are mutually exclusive. Either one must be set, otherwise the RPTR list is compromised and data access will fail.

RPTR_MTY indicates an RRDS empty slot

When RPTR_DIS is set, the RPTR addresses a Displaced Record Pointer, rather than the actual data. The format of a Displaced Record Pointer is as follows:

Label	Offset	Field type	Function
ZVSAMDRP		DSECT	Displaced Record Pointer
DRPIXLRA	X'000'	XL8	Indirect XLRA = location of actual record data

Free Space

Free space on any block is maintained in a single extent, usually but not necessarily following the RPTR list and preceding the stored record data.

Prefix Area Structure

The prefix area occurs only on the prefix block. It is located directly after the block header on the prefix block. No record pointer list lies in between.

The prefix area contains basic structural information about the file. It has the following format:

Label	Offset	Field type	Function
ZVSAMPFX		DSECT	Prefix area
PFXEYE	X'000'	CL4	Eye catcher
PFXZPFX		=C'zPFX'	
PFXRCLEN	X'004'	XL4	record length, max length if variable
PFXKYLEN	X'008'	XL4	key length
PFXKYOFF	X'00C'	XL4	key offset, excluding SPX/RDW
PFXDVOL@	X'010'	XL2	offset to the data component's volume label
PFXDNAM@	X'012'	XL2	offset to the data component's filename
PFXDPAT@	X'014'	XL2	offset to the data component's pathname
PFXXVOL@	X'016'	XL2	offset to the index component's volume label
PFXXNAM@	X'018'	XL2	offset to the index component's filename
PFXXPAT@	X'01A'	XL2	offset to the index component's pathname
PFXIXLVL	X'01C'	XL1	nr of index levels
PFXALTHR	X'01D'	XL1	allocation redrive threshold
PFXBLKSZ	X'01E'	XL4	blocksize used for this file (except prefix block)
PFXHXLRA	X'022'	XL8	XLRA of highest allocated block
PFXBMAP	X'02A'	XL8	XLRA of first spacemap block
PFXEMAP	X'032'	XL8	XLRA of last spacemap block
PFXMAPNW	X'03A'	XL8	XLRA of spacemap block last used for allocation
PFXBDATA	X'042'	XL8	XLRA of first data block
PFXEDATA	X'04A'	XL8	XLRA of last data block
PFXBSEGM	X'052'	XL8	XLRA of first segment block
PFXESEGM	X'05A'	XL8	XLRA of last segment block
PFXROOT	X'062'	XL8	XLRA of root index block
PFXBLVL0	X'06A'	XL8	XLRA of Header Block index level 0
PFXELVL0	X'072'	XL8	XLRA of End Block index level 0
PFXBLVL1	X'07A'	XL8	XLRA of Header Block index level 1
PFXELVL1	X'082'	XL8	XLRA of End Block index level 1

Label	Offset	Field type	Function
PFXBLVL2	X'08A'	XL8	XLRA of Header Block index level 2
PFXELVL2	X'092'	XL8	XLRA of End Block index level 2
PFXBLVL3	X'09A'	XL8	XLRA of Header Block index level 3
PFXELVL3	X'0A2'	XL8	XLRA of End Block index level 3
PFXBLVL4	X'0AA'	XL8	XLRA of Header Block index level 4
PFXELVL4	X'0B2'	XL8	XLRA of End Block index level 4
PFXBLVL5	X'0BA'	XL8	XLRA of Header Block index level 5
PFXELVL5	X'0C2'	XL8	XLRA of End Block index level 5
PFXBLVL6	X'0CA'	XL8	XLRA of Header Block index level 6
PFXELVL6	X'0D2'	XL8	XLRA of End Block index level 6
PFXBLVL7	X'0DA'	XL8	XLRA of Header Block index level 7
PFXELVL7	X'0E2'	XL8	XLRA of End Block index level 7
PFXBLVL8	X'0EA'	XL8	XLRA of Header Block index level 8
PFXELVL8	X'0F2'	XL8	XLRA of End Block index level 8
PFXBLVL9	X'0FA'	XL8	XLRA of Header Block index level 9
PFXELVL9	X'102'	XL8	XLRA of End Block index level 9
PFXBLVLA	X'10A'	XL8	XLRA of Header Block index level 10
PFXELVLA	X'112'	XL8	XLRA of End Block index level 10
PFXBLVLB	X'11A'	XL8	XLRA of Header Block index level 11
PFXELVLB	X'122'	XL8	XLRA of End Block index level 11
PFXBLVLC	X'12A'	XL8	XLRA of Header Block index level 12
PFXELVLC	X'132'	XL8	XLRA of End Block index level 12
PFXBLVLD	X'13A'	XL8	XLRA of Header Block index level 13
PFXELVLD	X'142'	XL8	XLRA of End Block index level 13
PFXBLVLE	X'14A'	XL8	XLRA of Header Block index level 14
PFXELVLE	X'152'	XL8	XLRA of End Block index level 14
PFXBLVLF	X'15A'	XL8	XLRA of Header Block index level 15
PFXELVLF	X'162'	XL8	XLRA of End Block index level 15
PFXMAPOF	X'16A'	XL3	offset within spacemap block to last used byte for allocation
PFXDTSKC	X'16D'	XL8	STCK of data component creation
PFXIXSKC	X'175'	XL8	STCK of index component creation
PFXDTSKU	X'17D'	XL8	STCK of last update to data component
PFXIXSKU	X'185'	XL8	STCK of last update to index component
PFXMAPDT	X'18D'	XL4	unsigned packed date YYYYMMDD of last allocation action
PFXFRSPC	X'195'	XL1	initial freespace % within block

Label	Offset	Field type	Function
PFXFRBLK	X'196'	XL2	initial freespace blocks
PFXFRINT	X'198'	XL2	initial freespace interval between free blocks
PFXFFLGS	X'19A'	XL1	file flags
PFX_ESDS		=X'80'	ESDS
PFX_KSDS		=X'40'	KSDS
PFX_RRDS		=X'20'	RRDS
PFX_LDS		=X'10'	LDS
PFX_AIX		=X'08'	AIX
PFX_INDX		=X'01'	index component
PFXRFLGS	X'19B'	XL1	record flags
PFX_RFIX		=X'80'	1=fixed, 0=variable
PFX_RSPN		=X'40'	1=spanned, 0=non-spanned
PFX_KUNQ		=X'20'	1=AIX unique, 0=AIX non-unique
PFX_AIXT		=X'10'	1=AIX on KSDS, 0=AIX on ESDS
PFXAIXN	X'19C'	X	no. of AIX's on the upgrade set
	X'19D'	XL3	reserved

There are 7 pointer fields in the prefix area. These point to fields allocated elsewhere in the prefix block. Their exact addresses on the prefix block may vary.

The PFXDVOL@, PFXDPAT@, PFXDNAM@ pointers and the PFXXVOL@, PFXXPAT@, PFXXNAM@ all point to a halfword-prefixed string.

Counters Area Structure

The counters area occurs only on the prefix block. It follows the prefix area and is doubleword aligned

The counters area has the following format:

Label	Offset	Field type	Function
ZVSAMCTR		DSECT	Counters area
CTREYE	X'000'	CL4	Eyecatcher
CTRZCTR		= 'zCTR'	
CTRAVGRL	X'004'	XL4	average record length
CTRAVSPAC	X'008'	XL8	available space
CTRHALCRBA	X'010'	XL8	high-allocated RBA
CTRENDRBA	X'018'	XL8	high water mark for the component
CTRNCIS	X'020'	XL8	nr of block-split operations
CTRNDEL	X'028'	XL8	nr of delete operations
CTRNECP	X'030'	XL8	nr of I/O operations
CTRNEXT	X'038'	XL8	nr of physical files allocated (always 1)
CTRNINSR	X'040'	XL8	nr of insert operations
CTRNLOGR	X'048'	XL8	nr of records in this component
CTRNRETR	X'050'	XL8	nr of retrieval operations
CTRNNUIW	X'058'	XL8	nr of zVSAM writes
CTRNUPDR	X'060'	XL8	nr of updates
CTRSDDTA	X'068'	XL8	uncompressed data size
CTRSTMST	X'070'	XL8	system timestamp of last close operation
CTRNUIW	X'078'	XL8	nr of user writes
CTRLOKEY@	X'080'	XL3	pointer to lowest valid key value
	X'083'	XL5	Reserved

The values in the counters area are maintained to support SHOWCB ACB and/or TESTCB ACB requests. They are mapped as follows:

Label	Keyword	SHOWCB	TESTCB
CTRAVSPAC	AVSPAC	ACB	ACB
CTRHALCRBA	HALCRBA	ACB	n.a.
CTRENDRBA	ENDRBA	ACB	ACB
CTRNCIS	NCIS	ACB	ACB
CTRNDCLR	NCLR	ACB	ACB
CTRNECP	NEXCP	ACB	ACB
CTRNEXT	NEXT	ACB	ACB
CTRNINSR	NINSR	ACB	ACB
CTRNLOGR	NLOGR	ACB	ACB
CTRNRETR	NRETR	ACB	ACB
CTRNNUIW	NUIW	ACB	n.a.
CTRNUPDR	NUPDR	ACB	ACB
CTRSOTA	SDTASIZE	ACB	n.a.
CTRSTMST	STMST	ACB	ACB
CTRNUIW	UIW	ACB	n.a.
CTRLOKEY@	LOKEY	ACB	n.a.

Spacemap Area Structure

The spacemap area fills every spacemap block. It consumes the entire area between the block header and the block footer. No free space is left.

The spacemap area is formatted as follows:

Label	Offset	Field type	Function
ZVSAMMAP		DSECT	Spacemap area
MAPXLRA	X'000'	XL8	XLRA of first block addressed by this spacemap area
MAPBITS	X'008'	0B	Bitmap indicating availability

The MAPBITS label addresses an array of bytes, each of which addresses 4 blocks of the file, the status of each block being represented by two bits. Each byte relates to 4 blocks in direct succession to one another, the bytes in the array mapping to successive sequences of 4 blocks.

The bits in the MAPBITS array are encoded as follows:

Value	Meaning
B'00'	block is not allocated
B'01'	block is allocated but may have insufficient free space. i.e. last allocation attempt failed, but a smaller record might fit. Or last allocation succeeded but left fewer than CTRAVGRL bytes of free space. Not used for blocks holding a segment of a segmented record.
B'10'	block is allocated and eligible for record allocation. i.e. last allocation succeeded and left enough free space for a record of average size. (CTRAVGRL) Not used for blocks holding a segment of a segmented record.
B'11'	block is full, blocks holds a segment of a segmented record, or block is an ELIX or spacemap block

Fixed Record Structure

Variable Record Structure

Fixed Segmented Record Structure

Variable Segmented Record Structure

Logical processes for ACB-based requests

ACB

GENCB ACB

MODCB ACB

SHOWCB ACB

TESTCB ACB

EXLST

GENCB EXLST

MODCB EXLST

SHOWCB EXLST

TESTCB EXLST

OPEN

Open logic has two major components: the open macro and the actual run-time logic to execute a request to open a file or a number of files.

OPEN macro logic

The open macro generates an open/close parameter list and/or an SVC 19 instruction to invoke the open routine. The syntax of the open macro is given [here](#)

The macro generates the following code:

MF variant	Generated Code
MF=L	Open/close parameter list data only
MF=E	1) Code to modify/populate the open/close parameter list at the indicated address, which may be a relocatable constant or a (register). 2) Code to invoke the open routine with R1 pointing to the open/close parameter list.
MF=I or omitted	1) Open/close parameter list data (inline) 2) Code to invoke the open routine with R1 pointing to the open/close parameter list.

When constructing the open/close parameter list, the end-of-list indicator of the last entry is set.

Open/close parameter list entries have two different formats. The one being used depends on the current addressing mode as indicated in the &MODE parameter.

When &MODE=24 then each entry is one fullword on a fullword boundary.

When &MODE=31 then each entry is two fullwords on a fullword boundary.

Label	Offset	Field type	Function
OCPL		DSECT	Open/Close Parameter List
OC24	X'000'	XL4	Entry for &MODE=24
OC24OPT	X'000'	XL1	Option byte (ignored for ACB)
OC24_EOL		X'80'	End-of-list indicator
OC24DCB	X'001'	AL3	Pointer to DCB or ACB
OC31	X'000'	XL8	Entry for &MODE=31
OC31OPT	X'000'	XL1	Option byte (ignored for ACB)
OC31_EOL		X'80'	End-of-list indicator
OC31NULL	X'001'	XL3	Reserved, should contain zeroes
OC31DCB	X'004'	AL4	Pointer to DCB or ACB

To invoke the open execution logic SVC 19 (X'13') is used. The type of parameter list is indicated as follows: if R1 is zero, then R0 contains a pointer to a list of OC31 entries, otherwise R1 contains a pointer to a list of OC24 entries.

OPEN execution logic

OPEN execution logic is implemented as a Java routine.

This logic consists of the following elements:

Action	Details
Determine type of parameter list	The list consists solely of OC31 entries, addressed by R0, if R1 = 0. The list consists solely of OC24 entries, addressed by R1, if R1 $\neq$ 0.
loop over all entries in the parameter list	End-of-list is indicated in the option byte of the entry
- check pointer: ACB or DCB	First byte = X'A0' => ACB V1 First four bytes = C'zACB' => assume ACB Otherwise => Assume DCB
- if DCB invoke DCB open routine	
- if ACB validate ACB	ACBID $\neq$ X'A0' => Error ACBSTYP $\neq$ X'10' => Error ACBVER $\neq$ X'02' => Error
- if ACB valid invoke VSAM open routine	
- next entry or end-of-loop	If bit 0 of OCPL entry is on, terminate loop

OPEN logic for DCB is beyond the scope of this document.

OPEN logic for ACB handles a single ACB and proceeds as follows:

Action	Details
Check ACB status	If ACB already open, issue error and fail open
Copy ACB to newly created FCB	FCB is the java-equivalent of the ACB
Extract DDNAME	Copy ACBDDNAM field from ACB/FCB
Find actual file name	Retrieve host variable with name matching ACBDDNAM content if not available: issue error and fail open
Validate against catalog	Find the file name in the catalog if missing: issue error
Issue OS open against file	Read-only if ACB specifies read-only for input Update/extend otherwise If unsuccessful issue error and fail open
Allocate buffer for prefix block	Save buffer address in FCB
Read first 4096 bytes into buffer	If read fails, issue error

Action	Details
Validate block header and footer	if BHDREYE <> C'HDR' issue error if BFTREYE <> C'FTR' issue error if BHDRSEQ# <> BFTRSEQ# issue error if BHDRVER <> X'02' issue error if BHDRSELF <> foxes issue error if BHDRPREV <> foxes issue error if BHDRNEXT <> foxes issue error if BHDRFLGS <> X'80' issue error
Validate prefix area	if PFXEYE <> C'zPFX' issue error if filename <> PFXDNAM issue error if file's path <> PFXDPAT issue error if PFX_INDIX is on issue error
Validate counters area	if CTREYE <> C'zCTR' issue error
Validate prefix against catalog	Only if no errors detected thus far: compare cluster type compare lrecl compare blocksize compare key offset compare key length
Fail open on error	If any error was detected: - request OS to close the file - free the prefix buffer - set buffer pointer in FCB to zeros - fail the open request
Issue OS open against index file	If PFXNAM@ is non-zero then open the indicated index file; read-only if ACB specifies read-only for input/update/extend otherwise. Read index header block and repeat all validations with the following modifications: - if PFX_INDIX is off rather than on issue an error
Fail open on error	If any error was detected: - request OS to close the files - free the prefix buffers - set buffer pointer in FCB to zeroes - fail the open request
Create data buffers	Based on ACBBUFND
Create index buffers	Only if index is present At least ACBBUFNI in total Exactly one for the root block At least 4 for each other index level
Open base cluster	If DDNAME refers to a path, repeat the open process for the base cluster. File names and other info to be gathered from the path's catalog entry. This basically repeats the steps above. We create an FCB – even though there is no ACB – to represent the cluster.

CLOSE

Upon close the prefix counter field CTRSTMST must be updated.

Do we need to include a last-updated timestamp to validate that last update of AIX is not behind the base cluster?

Close is a lot more complex than you realise, you have to check that there are no outstanding block/record locks

>>>include a last-updated timestamp to validate that last update of AIX is not behind the base cluster?

I'm not sure about the usefulness of timestamps anyway, if you want to maintain them...ok

An update of a base record can occur without affecting any AIX on the upgrade set, so only the timestamp of the base is affected. When AIX keys are changed then the base timestamp will precede the AIX

timestamp(s). Looks like a lot of work for no good purpose

Logical processes for RPL-based requests

RPL macro

GENCB RPL macro

MODCB RPL macro

SHOWCB RPL macro

TESTCB RPL macro

POINT macro

Prefix counter field CTRNEXCP needs to be incremented whenever a block is needed that does not yet reside in a buffer. If buffers need to be written out to make room for a block that needs to be read, then the CTRNEXCP counter needs to be incremented as well.

Prefix counter field CTRNRETR needs to be incremented for every block accessed, whether it needs to be read, or already resides in a buffer does not matter for this count field.

If buffers need to be written out in order to allocate a buffer for a block that needs to be read, then prefix counter CTRNNUIW needs to be incremented.

GET macro

Prefix counter field CTRNEXCP needs to be incremented whenever a block is needed that does not yet reside in a buffer. If buffers need to be written out to make room for a block that needs to be read, then the CTRNEXCP counter needs to be incremented as well.

Prefix counter field CTRNRETR needs to be incremented for every block accessed, whether it needs to be read, or already resides in a buffer does not matter for this count field.

If buffers need to be written out in order to allocate a buffer for a block that needs to be read, then prefix counter CTRNNUIW needs to be incremented.

PUT macro

Prefix counter field CTRAVSPAC must be updated. When consuming free space to allocate a new record, reduce CTRAVSPAC with space consumed. When allocating a new block (or splitting an existing block) add the blocksize – (block header, block footer, RPTR area) and reduce with amount used up (record length, including RDW/SPX and RPTR). When lengthening a record, subtract the difference; when shortening a record, add the difference.

Prefix counter fields CTRHALCRBA and CTRENDRBA should be updated whenever a record is added beyond the current value of these fields. CTRENDRBA also should be updated in case the last record in the component is lengthened.

Prefix counter field CTRNCIS should be incremented whenever a block needs to be split.

Prefix counter field CTRNEXCP needs to be incremented whenever a block needs to be written out. This occurs when the cluster was opened with MACRF=NDF. With MACRF=DFR no writes are forced and no EXCP needs to be counted.

When a file can no longer be extended a secondary file needs to be allocated. When that is done transparently, CTRNEXT needs to be incremented. Setting its initial value is a function of zREPRO.

Prefix counter fields CTRNINSR and CTRNLOGR need to be incremented whenever a new record is added to the component.

Prefix counter field CTRNRETR needs to be incremented for every block accessed, whether it needs to be read, or already resides in a buffer does not matter for this count field. For basic put operation this is irrelevant, but for updating an index or an AIX reads may be needed and should be counted for the component being read.

When an existing record is updated, then CTRNUPDR needs to be incremented.

When adding a record the record length (including SPX/RDW, but excluding RPTR) needs to be added to the prefix counter field CTRSDTA. When a record is updated to a different length, then the difference in length needs to be accounted into the CTRSDTA field.

When a user write is forced then the prefix counter field CTRNUIW needs to be incremented. This happens when a put is issued to a cluster that was opened with MACRF=NDF. For clusters opened with MACRF=DFR writing is done by zVSAM when the buffer is needed for a different block. These writes are counted in the CTRNNUIW field.

The prefix counter field CTRAVGRL contains the value of CTRSDAT / CTRNLOGR rounded up to nearest integer. The field should be updated whenever either or both of the input values are changed.

The prefix counter field CTRLOKEY@ points to a value of PFXKYLEN bytes. The value should be updated whenever a record is inserted that has a lower key than the current lowest key.

ERASE macro

Prefix counter field CTRNDELRL should be incremented for every successful erase operation.

Prefix counter field CTRNEXCP needs to be incremented whenever a block is needed that does not yet reside in a buffer. If buffers need to be written out to make room for a block that needs to be read, then the CTRNEXCP counter needs to be incremented as well.

Prefix counter field CTRNLOGR needs to be decremented for every successful erase operation.

Prefix counter field CTRNRETR needs to be incremented for every block accessed, whether it needs to be read, or already resides in a buffer does not matter for this count field. For basic erase operation this is irrelevant, but for updating an index or an AIX reads may be needed and should be counted for the component being read.

When erasing a record the record length (including SPX/RDW, but excluding RPTR) needs to be subtracted from the prefix counter field CTRSDTA.

When a user write is forced then the prefix counter field CTRNUIW needs to be incremented. This happens when an erase is issued to a cluster that was opened with MACRF=NDF. For clusters opened with MACRF=DFR writing is done by zVSAM when the buffer is needed for a different block. These writes are counted in the CTRNNUIW field.

The prefix counter field CTRAVGRL contains the value of CTRSDAT / CTRNLOGR rounded up to nearest integer. The field should be updated after every successful erase operation.

The prefix counter field CTRLOKEY@ points to a value of PFXKEYLN bytes. The value should be updated whenever the record is erased with the current lowest key.

CHECK macro

ENDREQ macro

VERIFY macro

Prefix counter field CTRNEXCP needs to be incremented whenever a block is needed that does not yet reside in a buffer.

Addenda

API for ACB-based interfaces

ACB Macro parameters

With the exception of the DDNAME parameter, all supported parameters are implemented compatibly with IBM's VSAM implementation. For details, please refer to the relevant IBM manual.

For ease of access a short summary follows here:

AM= Optional parameter. AM=VSAM is the default. No other values are supported.

DDNAME= DDNAME is required before open is executed. If DDNAME is not supplied on the ACB macro, the label used on the ACB macro is used as DDNAME.
If neither is specified, a proper value must be supplied by using MODCB ACB.
In zVSAM the DDNAME refers to the name of an environment variable in the host OS.
This variable in turn should contain the path and qualified filename of the cluster to be opened. The qualifier is the name of an environment variable in the host OS and is the path to the assembled catalog. For more information on zVSAM catalogs, please refer to the "z390_zVSAM_Catalog_User_Guide".

PASSWD= Supply the address of the password, consisting of a single byte with the password's length (1-8 characters) followed by the password value.

EXLST= Pointer to an exit list. Please see the EXLST macro description for details.

MACRF= List of keywords specifying how the cluster will be processed after open.
Please see below for the list of supported keywords and their meaning.

BUFSP= Maximum buffer space in virtual storage for this cluster. This is the combined size in bytes of all buffers allocated for this cluster. If (BUFND + BUFNI) * Block_size exceeds the value specified for BUFSP, then BUFND and BUFNI will be reduced proportionally to keep total allocation below the limit specified in the BUFSP parameter.

BUFND= Number of data buffers to allocate. Specify a number up to 65535.
When over-allocating (see BUFSP parameter above) fewer data buffers will be allocated than requested.

BUFNI= Number of index buffers to allocate. Specify a number up to 65535.
When over-allocating (see BUFSP parameter above) fewer index buffers will be allocated than requested.

RMODE31= Specifies whether buffers and/or control blocks should be allocated below the 16M line, or may be allocated above the 16M line as follows:
NONE Control Blocks and buffers below 16M
CB Control Blocks above or below 16M, buffers below 16M
BUFF Control Blocks below 16M, buffers above or below 16M
ALL Control Blocks and buffers above 16M or below 16M
The default for RMODE31 is NONE

STRNO=	Maximum number of strings (concurrent requests) for this ACB. If specified requires a number between 1 and 255
BSTRNO=	Beginning (or initial) number of strings (concurrent requests) allocated to this ACB when a path is opened. If specified requires a number between 1 and 255
ADR/KEY	Non-exclusive MACRF keywords indicating whether the cluster may be accessed by address or by key. ADR can be used only with ESDS or KSDS to access records by RBA or XRBA. KEY can be used with KSDS to access records by key KEY can be used with RRDS to access records by relative record number.
DFR/NDF	Mutually exclusive MACRF keywords indicating whether or not buffer changes need to be written out to the file immediately. DFR allows VSAM to defer writing and keep changes in the buffer. When multiple changes are combined, fewer I/Os are needed which should improve program performance. NDF disallows VSAM to defer writing, forcing a buffer write for every change to the buffer.
DIR	MACRF keyword indicating that the cluster will be processed directly. DIR can be used with ESDS to access data randomly DIR can be used with KSDS to access data randomly DIR can be used with RRDS to access data randomly
SEQ	MACRF keyword indicating that the cluster will be processed sequentially. SEQ can be used with ESDS to access data sequentially SEQ can be used with KSDS to access data sequentially SEQ can be used with RRDS to access data sequentially SEQ serves as a default among the subgroup DIR/SEQ/SKP, which are otherwise independent MACRF keywords.
SKP	MACRF keyword to allow skip-sequential access. i.e. specifying this keyword allows usage of the POINT macro to position the file to a specific position to access data randomly. SKP can be used with KSDS to randomly position the file to a specific key prior to sequential access SKP can be used with RRDS to randomly position the file to a specific RRN prior to sequential access
IN/OUT	Non-exclusive MACRF keywords indicating whether the cluster will be processed for input only or for both input and output. OUT implies IN. IN data in the cluster can be read but not changed in any way OUT data in the cluster can be read, updated, inserted, or deleted.
NIS/SIS	Mutually exclusive MACRF keywords indicating how zVSAM inserts new records into the cluster. Relevant only for KSDS clusters. NIS Normal insert strategy: zVSAM will insert records optimizing for inserts that are dispersed randomly across the data set SIS Sequential insert strategy: zVSAM will insert records optimizing for inserts that are (generally, mostly) packed together in a sequential manner.

NRM/AIX Mutually exclusive MACRF keywords indicating how zVSAM is to process the DDNAME

NRM Normal mode: zVSAM uses the DDNAME to access the dataset directly

AIX AIX mode: zVSAM will use a path to access an AIX directly, rather than using it to
 access records in the underlying base cluster

GENCB ACB macro parameters

All supported parameters are implemented compatibly with IBM's VSAM implementation. For details, please refer to the relevant IBM manual.

For ease of access a short summary follows here:

ACB=addr required to indicate the ACB-supporting logic of the macro is being invoked

COPIES= Number of identical copies to generate. The default is 1.
Specify a number between 1 and 65535

WAREA= Address of a work area where the ACB/ACBs is/are to be constructed.
When WAREA is specified, LENGTH must be specified too.
When WAREA is not specified, the CBMR handler allocates an area of storage.
The address of this area is returned in R1; its length in R0.

LENGTH= Length in bytes of the area indicated by WAREA.
When LENGTH is specified, WAREA must be specified as well.

LOC= Where a work area for constructing the ACB/ACBs is to be allocated.
Used only when WAREA and LENGTH are not specified.
Supports the keywords BELOW and ANY, with BELOW being the default
BELOW the work area is to be allocated below the line
ANY the work area is to be allocated above the line if possible,
 below the line otherwise.

other Any parameters and/or keywords supported by the ACB macro.
Please see the description of the ACB macro for details.
Supported parameters and keywords on the ACB macro are supported on
GENCB ACB as well. Likewise, unsupported parameters and keywords on the ACB
macro are not supported on GENCB ACB either.
How the parameters can be specified differs per parameter. For a complete list of
options, please see the IBM manual "DFSMS Macro Instructions for Data Sets"
or equivalent for the operating system and version that you are porting to/from.
Please note: not supported are expressions like (S,scon) or (*,scon)

MF=I or omitted Specifies the standard form of the GENCB to generate an inline CBMR and an inline call to
the CBMR handler.

MF=L Specifies the list form of the GENCB macro which generates an inline CBMR but no call to
the CBMR handler.

MF=(L,addr) Specifies the list form of the GENCB macro to generate a remote CBMR at the indicated
location. No call to the CBMR handler is generated.

MF=(L,addr,label) Same as MF=(L,addr) but label will be equated to the length of the CBMR.

MF=(E,addr) Specifies the execute form of the GENCB macro to generate code that will dynamically
modify the CBMR at the indicated address according to the parameters specified before
calling the CBMR handler.

MF=(G,addr) Specifies the generate form of the GENCB macro to generates code to modify the indicated CBMR as specified by the other parameters and to call the CBMR handler.

MF=(G,addr,label)

Same as MF=(G,addr) but label will be equated to the length of the CBMR

MODCB ACB macro parameters

All supported parameters are implemented compatibly with IBM's VSAM implementation. For details, please refer to the relevant IBM manual.

For ease of access a short summary follows here:

ACB=addr required to indicate the ACB to be modified

other Any parameters and/or keywords supported by the ACB macro.
Please see the description of the ACB macro for details.
Supported parameters and keywords on the ACB macro are supported on MODCB ACB as well. Likewise, unsupported parameters and keywords on the ACB macro are not supported on MODCB ACB either.
How the parameters can be specified differs per parameter. For a complete list of options, please see the IBM manual “DFSMS Macro Instructions for Data Sets” or equivalent for the operating system and version that you are porting to/from.
Please note: not supported are expressions like (S,scon) or (*,scon)

MF=I or omitted

Specifies the standard form of the MODCB to generate an inline CBMR and an inline call to the CBMR handler.

MF=L

Specifies the list form of the MODCB macro which generates an inline CBMR but no call to the CBMR handler.

MF=(L,addr) Specifies the list form of the MODCB macro to generate a remote CBMR at the indicated location. No call to the CBMR handler is generated.

MF=(L,addr,label)

Same as MF=(L,addr) but label will be equated to the length of the CBMR.

MF=(E,addr) Specifies the execute form of the MODCB macro to generate code that will dynamically modify the CBMR at the indicated address according to the parameters specified before calling the CBMR handler.

MF=(G,addr) Specifies the generate form of the MODCB macro to generates code to modify the indicated CBMR as specified by the other parameters and to call the CBMR handler.

MF=(G,addr,label)

Same as MF=(G,addr) but label will be equated to the length of the CBMR

SHOWCB ACB macro parameters

All supported parameters are implemented compatibly with IBM's VSAM implementation. For details, please refer to the relevant IBM manual.

For ease of access a short summary follows here:

ACB=addr required to indicate the ACB to be queried

Each keyword specified returns a field of 4 or 8 bytes. These return values are stored consecutively in the return area specified in the AREA= and LENGTH= parameters. Some keywords are valid only when the ACB is open. An error is returned when any of these keywords are used while the ACB is not open.

For some keywords there are a few things to keep in mind:

AVSPAC – Count of available space in bytes – taken from prefix counter field CTRAVSPAC

BFRFND – nr of times since this ACB was opened that a get/read request for this ACB was satisfied from a buffer, without doing any I/O.

BUFRDS – nr of times since this ACB was opened an I/O was needed for this ACB to read block into a buffer.

CDTASIZE – Compressed data size. Since zVSAM does not support compression, this is the same as SDTASIZE. Taken from prefix counter field CTRSDTA

ENDRBA – high water mark of the component in bytes, discounting the prefix block.
Taken from prefix counter field CTRENRBA

FS – For data component $\text{PFXFRBLK} / (\text{PFXFRBLK} + \text{PFXFRINT}) * 100$. Foxes for index.

HALCRBA – RBA of highest allocated record. (is this current highest, or highest ever?)
Taken from prefix counter field CTRHALCRBA.

KEYLEN – length of key field. For KSDS this is the length of the key field and is taken from prefix field PFXKEYLEN. For RRDS/ESDS the value is always 8.

NCIS – nr of times a block was split for this component. Taken from prefix counter field CTRNCIS.

NDELRL – nr of times a record was deleted for this component.
Taken from prefix counter field CTRNDELRL.

NEXCP – nr of times an I/O was issued for this component. Taken from prefix counter field CTRNEXCP.

NEXT – nr of extents. For zVSAM this is the number of files used to store the component.
Taken from prefix counter field CTRNEXT.

NINSRL – nr of times a record was inserted for this component.
Taken from prefix counter field CTRNINSRL.

NIXL – nr of index levels. Taken from prefix field PFXIXLVL.

NLOGR – nr of records in the component. Taken from prefix counter field CTRNLOGR.

- NRETR – nr of times a record was retrieved for this component.
Taken from prefix counter field CTRNRETR.
- NSSS – nr of CA splits. Always foxes.
- NUIW – nr of times a block was written for this component by zVSAM rather than the user program.
Taken from prefix counter field CTRNNUIW.
- NUPDR – nr of times a record was updated for this component.
Taken from prefix counter field CTRNUPDR.
- RKP – relative key position. Taken from prefix field PFXKYOFF.
- SDTASIZE – Total nr of data bytes currently stored in the component. Sum of all record lengths.
Taken from prefix counter field CTRSDTA.
- SHRPOOL - Shared pool number. Taken from ACBSHRP
- STMST – system timestamp of last close operation on the component.
Taken from prefix counter field CTRSTMST
- UIW – nr of times a block was written for this component by the user program rather than zVSAM
Taken from prefix counter field CTRNUIW.
- MF=I or omitted
Specifies the standard form of the SHOWCB to generate an inline CBMR and an inline call to the CBMR handler.
- MF=L
Specifies the list form of the SHOWCB macro which generates an inline CBMR but no call to the CBMR handler.
- MF=(L,addr) Specifies the list form of the SHOWCB macro to generate a remote CBMR at the indicated location. No call to the CBMR handler is generated.
- MF=(L,addr,label)
Same as MF=(L,addr) but label will be equated to the length of the CBMR.
- MF=(E,addr) Specifies the execute form of the SHOWCB macro to generate code that will dynamically modify the CBMR at the indicated address according to the parameters specified before calling the CBMR handler.
- MF=(G,addr) Specifies the generate form of the SHOWCB macro to generates code to modify the indicated CBMR as specified by the other parameters and to call the CBMR handler.
- MF=(G,addr,label)
Same as MF=(G,addr) but label will be equated to the length of the CBMR

TESTCB ACB macro parameters

All supported parameters are implemented compatibly with IBM's VSAM implementation. For details, please refer to the relevant IBM manual.

For ease of access a short summary follows here:

ACB=addr required to indicate the ACB to be tested

All other keywords function the same way that they do on a SHOWCB ACB request. Please see the preceding chapter for details [here](#)

MF=I or omitted

Specifies the standard form of the TESTCB to generate an inline CBMR and an inline call to the CBMR handler.

MF=L

Specifies the list form of the TESTCB macro which generates an inline CBMR but no call to the CBMR handler.

MF=(L,addr)

Specifies the list form of the TESTCB macro to generate a remote CBMR at the indicated location. No call to the CBMR handler is generated.

MF=(L,addr,label)

Same as MF=(L,addr) but label will be equated to the length of the CBMR.

MF=(E,addr)

Specifies the execute form of the TESTCB macro to generate code that will dynamically modify the CBMR at the indicated address according to the parameters specified before calling the CBMR handler.

MF=(G,addr)

Specifies the generate form of the TESTCB macro to generate code to modify the indicated CBMR as specified by the other parameters and to call the CBMR handler.

MF=(G,addr,label)

Same as MF=(G,addr) but label will be equated to the length of the CBMR

EXLST macro parameters

All supported parameters are implemented compatibly with IBM's VSAM implementation.
For details, please refer to the relevant IBM manual.

For ease of access a short summary follows here:

AM= Optional parameter. AM=VSAM is the default. No other values are supported.

EODAD= Optional parameter to specify the entry address of an exit that handles an end-of-data condition during sequential access.
The routine address may be followed by a modifier. For details, please see below.
The AMODE for the routine is encoded in the address using the common convention.

LERAD= Optional parameter to specify the entry address of an exit that handles logic errors.
The routine address may be followed by a modifier. For details, please see below.
The AMODE for the routine is encoded in the address using the common convention.

SYNAD= Optional parameter to specify the entry address of an exit that handles physical errors.
The routine address may be followed by a modifier. For details, please see below.
The AMODE for the routine is encoded in the address using the common convention.

mod modifier, can optionally be specified after each routine address.
Values: A or N for Active or Not-active.
As long as the routine is not active it will not be called by zVSAM.
The secondary modifier of L (for Load from Linklib) is not supported.

GENCB EXLST macro parameters

All supported parameters are implemented compatibly with IBM's VSAM implementation.
For details, please refer to the relevant IBM manual.

For ease of access a short summary follows here:

EXLST=addr required to indicate the EXLST-supporting logic of the macro is being invoked

COPIES= Number of identical copies to generate. The default is 1.
Specify a number between 1 and 65535

WAREA= Address of a work area where the EXLST/EXLSTs is/are to be constructed.
When WAREA is specified, LENGTH must be specified too.
When WAREA is not specified, the CBMR handler allocates an area of storage.
The address of this area is returned in R1; its length in R0.

LENGTH= Length in bytes of the area indicated by WAREA.
When LENGTH is specified, WAREA must be specified as well.

LOC= Where a work area for constructing the EXLST/EXLSTs is to be allocated.
Used only when WAREA and LENGTH are not specified.
Supports the keywords BELOW and ANY, with BELOW being the default
BELOW the work area is to be allocated below the line
ANY the work area is to be allocated above the line if possible, below the line otherwise

other Any parameters and/or keywords supported by the EXLST macro.
Please see the description of the EXLST macro for details [here](#)
Supported parameters and keywords on the EXLST macro are supported on GENCB EXLST as well. Likewise, unsupported parameters and keywords on the EXLST macro are not supported on GENCB EXLST either.
How the parameters can be specified differs per parameter. For a complete list of options, please see the IBM manual “DFSMS Macro Instructions for Data Sets” or equivalent for the operating system and version that you are porting to/from.
Please note: not supported are expressions like (S,scon) or (*,scon)

MF=I or omitted
Specifies the standard form of the GENCB to generate an inline CBMR and an inline call to the CBMR handler.

MF=L Specifies the list form of the GENCB macro which generates an inline CBMR but no call to the CBMR handler.

MF=(L,addr) Specifies the list form of the GENCB macro to generate a remote CBMR at the indicated location. No call to the CBMR handler is generated.

MF=(L,addr,label)
Same as MF=(L,addr) but label will be equated to the length of the CBMR.

MF=(E,addr) Specifies the execute form of the GENCB macro to generate code that will dynamically modify the CBMR at the indicated address according to the parameters specified before calling the CBMR handler.

MF=(G,addr) Specifies the generate form of the GENCB macro to generates code to modify the indicated CBMR as specified by the other parameters and to call the CBMR handler.

MF=(G,addr,label)

Same as MF=(G,addr) but label will be equated to the length of the CBMR

MODCB EXLST macro parameters

All supported parameters are implemented compatibly with IBM's VSAM implementation. For details, please refer to the relevant IBM manual.

For ease of access a short summary follows here:

EXLST=addr required to indicate the EXLST to be modified

other Any parameters and/or keywords supported by the EXLST macro. Please see the description of the EXLST macro for details. Supported parameters and keywords on the EXLST macro are supported on MODCB EXLST as well. Likewise, unsupported parameters and keywords on the EXLST macro are not supported on MODCB EXLST either. How the parameters can be specified differs per parameter. For a complete list of options, please see the IBM manual “DFSMS Macro Instructions for Data Sets” or equivalent for the operating system and version that you are porting to/from.

Please note: not supported are expressions like (S,scon) or (*,scon)

MF=I or omitted

Specifies the standard form of the MODCB to generate an inline CBMR and an inline call to the CBMR handler.

MF=L

Specifies the list form of the MODCB macro which generates an inline CBMR but no call to the CBMR handler.

MF=(L,addr) Specifies the list form of the MODCB macro to generate a remote CBMR at the indicated location. No call to the CBMR handler is generated.

MF=(L,addr,label)

Same as MF=(L,addr) but label will be equated to the length of the CBMR.

MF=(E,addr) Specifies the execute form of the MODCB macro to generate code that will dynamically modify the CBMR at the indicated address according to the parameters specified before calling the CBMR handler.

MF=(G,addr) Specifies the generate form of the MODCB macro to generate code to modify the indicated CBMR as specified by the other parameters and to call the CBMR handler.

MF=(G,addr,label)

Same as MF=(G,addr) but label will be equated to the length of the CBMR

SHOWCB EXLST macro parameters

All supported parameters are implemented compatibly with IBM's VSAM implementation. For details, please refer to the relevant IBM manual.

For ease of access a short summary follows here:

EXLST=addr required to indicate the EXLST to be queried

Each keyword specified returns a field of 4 bytes. These return values are stored consecutively in the return area specified in the AREA= and LENGTH= parameters.

MF=I or omitted

Specifies the standard form of the SHOWCB to generate an inline CBMR and an inline call to the CBMR handler.

MF=L

Specifies the list form of the SHOWCB macro which generates an inline CBMR but no call to the CBMR handler.

MF=(L,addr)

Specifies the list form of the SHOWCB macro to generate a remote CBMR at the indicated location. No call to the CBMR handler is generated.

MF=(L,addr,label)

Same as MF=(L,addr) but label will be equated to the length of the CBMR.

MF=(E,addr)

Specifies the execute form of the SHOWCB macro to generate code that will dynamically modify the CBMR at the indicated address according to the parameters specified before calling the CBMR handler.

MF=(G,addr)

Specifies the generate form of the SHOWCB macro to generate code to modify the indicated CBMR as specified by the other parameters and to call the CBMR handler.

MF=(G,addr,label)

Same as MF=(G,addr) but label will be equated to the length of the CBMR

TESTCB EXLST macro parameters

All supported parameters are implemented compatibly with IBM's VSAM implementation.
For details, please refer to the relevant IBM manual.

For ease of access a short summary follows here:

EXLST=addr required to indicate the EXLST to be tested

mod modifier, can optionally be specified after each routine address.
 Values: A or N for Active or Not-active.
 When this modifier is specified, only Equal or Not-Equal condition can be returned.
 The secondary modifier of L (for Load from Linklib) is not supported.

MF=I or omitted

Specifies the standard form of the TESTCB to generate an inline CBMR and an inline call to the CBMR handler.

MF=L Specifies the list form of the TESTCB macro which generates an inline CBMR but no call to the CBMR handler.

MF=(L,addr) Specifies the list form of the TESTCB macro to generate a remote CBMR at the indicated location. No call to the CBMR handler is generated.

MF=(L,addr,label)

Same as MF=(L,addr) but label will be equated to the length of the CBMR.

MF=(E,addr) Specifies the execute form of the TESTCB macro to generate code that will dynamically modify the CBMR at the indicated address according to the parameters specified before calling the CBMR handler.

MF=(G,addr) Specifies the generate form of the TESTCB macro to generates code to modify the indicated CBMR as specified by the other parameters and to call the CBMR handler.

MF=(G,addr,label)

Same as MF=(G,addr) but label will be equated to the length of the CBMR

OPEN macro parameters

All supported parameters are implemented compatibly with IBM's VSAM implementation. For details, please refer to the relevant IBM manual.

For ease of access a short summary follows here:

- | | |
|-----------------|--|
| entry | The OPEN macro accepts a list of entries. Each entry consists of two consecutive parameters: an address and an optional list of options. |
| address | The address can be specified as an A-type address or as a register. If a register is coded the register number or name must be enclosed in parentheses. The address can be either the address of a DCB or the address of an ACB. |
| options | For a DCB options may be encoded according to the z390 File Access Method Guide . For an ACB the options list is ignored and should be coded as an omitted parameter. Any options (e.g. IN/OUT) are taken from the ACB, not the open parmlist. |
| MF=I or omitted | An open parmlist is generated inline, plus a call to the OPEN SVC using the parmlist. |
| MF=L | With MF=L an open parmlist is generated inline |
| MF=(L,addr) | Code is generated to construct the open parmlist at run-time, rather than at assembly time, at the indicated address.
If the address is specified within parentheses, it is assumed to indicate a register pointing to the desired address. |
| MF=(E,addr) | Code is generated to call the CLOSE SVC using the parmlist at the indicated address.
If the address is specified within parentheses, it is assumed to indicate a register pointing to the desired address. |

CLOSE macro parameters

All supported parameters are implemented compatibly with IBM's VSAM implementation. For details, please refer to the relevant IBM manual.

For ease of access a short summary follows here:

- | | |
|-----------------|---|
| entry | The CLOSE macro accepts a list of entries. Each entry consists of two consecutive parameters: an address and an optional list of options. |
| address | The address can be specified as an A-type address or as a register. If a register is coded the register number or name must be enclosed in parentheses. The address can be either the address of a DCB or the address of an ACB. |
| options | For a DCB options may be encoded according to the z390 File Access Method Guide . For an ACB the options list is ignored and should be coded as an omitted parameter. |
| MF=I or omitted | If the MF parameter is omitted a close parmlist is generated inline, plus a call to the CLOSE SVC using the parmlist. |
| MF=L | With MF=L a close parmlist is generated inline |
| MF=(L,addr) | Code is generated to construct the close parmlist at run-time, rather than at assembly time, at the indicated address.
If the address is specified within parentheses, it is assumed to indicate a register pointing to the desired address. |
| MF=(E,addr) | Code is generated to call the CLOSE SVC using the parmlist at the indicated address.
If the address is specified within parentheses, it is assumed to indicate a register pointing to the desired address. |

zACB description

The structure and layout of the zACB are not formally part of the interface and may change in future releases. Therefore the zACB layout is shown here only for the sake of completeness. Direct access to subfields in the zACB is discouraged. Use SHOWCB ACB, TESTCB ACB and/or MODCB ACB to inspect, test, and/or modify the zACB's content. Accessing subfields of the zACB directly may adversely impact portability of your programs.

Label	Equate	Designation	Remarks
IFGACB IHAACB		DSECT	
ACBEYE		CL4	Eye catcher
	ACBZACB	C'zACB'	Fixed value
ACBID		XL1	Identifier
	ACBIDVAL	X'A0'	ACB
ACBSTYP ACBSTYPE		XL1	Subtype
	ACBVSAM	X'10'	VSAM ACB
ACBLENG ACBLENG2 ACBLEN ACBLEN2		H	Length of this ACB in bytes
ACBDDNM ACBDDNAM		CL8	Name of host variable holding cluster name
ACBMACRF			ACBMACR1 + ACBMACR2
ACBMACR1		BL1	Option bits
	ACBKEY ACBMACR1_KEY	X'80'	Indexed access by logical key
	ACBADDR ACBADD ACBMACR1_ADR	X'40'	Non-indexed access by address
	ACBSEQ ACBMACR1_SEQ	X'10'	Sequential access
	ACBDIR ACBMACR1_DIR	X'08'	Direct access
	ACBIN ACBMACR1_IN ACBGET	X'04'	Get allowed
	ACNOUT ACBMACR1_OUT ACBPUT	X'02'	Get / Put / Erase allowed

Label	Equate	Designation	Remarks
IFGACB IHAACB		DSECT	
ACBMACR2		BL1	More option bits
	ACBSKP ACBMACR2_SKP	X'10'	Skip-sequential access
	ACBRST ACBMACR2_RST	X'04'	Reserved
	ACBAIX ACBMACR2_AIX ACBAIXP	X'01'	Access through AIX
ACBMACR3		BL1	Additional option bits
	ACBNLW	X'80'	Reserved
	ACBLSR ACBMACR3_LSR	X'40'	Reserved
	ACBGSR ACBMACR3_GSR	X'20'	Reserved
	ACBDFR ACBMACR3_DFR	X'08'	Deferred writes allowed
	ACBSIS ACBMACR3_SIS	X'04'	Sequential insert strategy
	ACBMODE	X'01'	Buffers allowed above the line
ACBMACR4		BL1	Reserved
ACBBUFND		XL2	Nr of data buffers
ACBBUFNI		XL2	Nr of index buffers
ACBBUFSP		F	Max buffer space in bytes
		H	Reserved
ACBLRECL		XL2	Record length
ACBPASSW		A	Password pointer
ACBEXLST ACBUFL		A	EXLST pointer
ACBINFLG ACBINFL			ACBINFL1 + ACBINFL2
ACBINFL1		BL1	Indicator flags
	ACBCAT	X'10'	Reserved
ACBINFL2		BL1	More indicator flags

Label	Equate	Designation	Remarks
IFGACB IHAACB		DSECT	
	ACBSWARN	X'80'	Suppress open warning
	ACBSHROP	X'03'	Reserved
	ACBSHR02	X'02'	Reserved
	ACBSHR01	X'01'	Reserved
ACBOFLGS		BL1	Open / Close flags
	ACBR31B	X'80'	31-bit addressing for buffers
	ACBR31C	X'40'	31-bit addressing for control blocks
	ACBEOV	X'20'	Reserved
	ACBOPEN	X'10'	ACB currently open
	ACBDSERR	X'08'	Error – ACB must be closed
	ACBRECOV	X'04'	Open for recovery
	ACBEXFG ACBLOCK	X'02'	Off when user exit in progress
	ACBIOFSG ACBBUSY	X'01'	Open / close in progress
ACBERFLG		BL1	Error flags
ACBBSTNO		XL1	Reserved
ACBSTRNO		XL1	Reserved
ACBSHRP		XL1	Reserved
ACBVER		XL1	zACB layout version
ACBV2		=X'02'	zACB V2 version
ACBPFX		AL4	Prefix Block in buffer
ACBXPFX		AL4	Prefix Block of index zeroes if no index open

zEXLST description

The structure and layout of the zEXLST are not formally part of the interface and may change in future releases. Therefore the zEXLST layout is shown here only for the sake of completeness. Direct access to subfields in the zEXLST is discouraged. Use SHOWCB EXLST, TESTCB EXLST and/or MODCB EXLST to inspect, test, and/or modify the zEXLST's content. Accessing subfields of the zEXLST directly may adversely impact portability of your programs.

Label	Equate	Designation	Remarks
IFGEXLST		DSECT	
EXLEYE		CL4	Eye catcher
	EXLZLST	C'zLST'	Fixed value
EXLLEN EXLLEN2		H	Length of exit list
EXLSTYP		XL1	Subtype
	EXLSVSAM	X'10'	zVSAM
EXLEODF		XL1	Eodad routine flags
	EXLEODS	X'80'	Present
	EXLEODA	X'40'	Active
EXLLERF		XL1	Lerad routine flags
	EXLLERS	X'80'	Present
	EXLLERA	X'40'	Active
EXLSYNF		XL1	Synad routine flags
	EXLSYNS	X'80'	Present
	EXLSYNA	X'40'	Active
EXLEODP		AL4	EODAD address
EXLLERP		AL4	SYNAD address
EXLSYNP		AL4	LERAD address

CBMR description

The structure and layout of the CBMR are not formally part of the interface and may change in future releases. Therefore the CBMR layout is shown here only for the sake of completeness. Direct access to subfields in the CBMR is discouraged. Use SHOWCB, TESTCB and/or MODCB to inspect, test, and/or modify the content of an ACB, EXLST, or RPL. Use the appropriate MF= parameter on any of these macros to modify and/or use a CBMR.

The CBMR consists of three parts: a header, a body, and a tail. The header has a fixed layout. The body consists of request-dependent fields and a list of verb codes. The tail contains all the data fields that go with the verb codes. Data fields can be 4 or 8 bytes in length. Verb codes '01'-X'DF' have a 4-byte data field. Verb codes X'E0'-X'FF' have an 8-byte data field. All data fields in the tail are allocated consecutively, in the same order as the verbs that define their meaning.

For valid verb values and their data field lengths, please refer to the relevant CBMR body chapter.

Label	Equate	Designation	Remarks
CBMR		DSECT	
CBMREYE		CL4	Eye catcher
	CBMRCBMR	C'CBMR'	Fixed value
CBMRREQ		XL1	Request type
	CBMRACB	X'01'	ACB request
	CBMRXLST	X'02'	EXLST request
	CBMRRPL	X'04'	RPL request
	CBMRGEN	X'10'	GENCB request
	CBMRMOD	X'20'	MODCB request
	CBMRSHOW	X'40'	SHOWCB request
	CBMRTEST	X'80'	TESTCB request
CBMRRMOD		XL1	Request modifier
	CBMRLOCB	X'01'	Work area below 16M
	CBMRLOCA	X'03'	Work area below 2G
	CBMROBJD	X'10'	Object=Data
	CBMROBJI	X'20'	Object=Index
CBMRVRBS		AL1	Nr of verbs in body
CBMRWORK		AL4	Workarea pointer
CBMRWLEN		AL2	Workarea length
CBMRSIZE		AL2	CBMR size
CBMRBODY		Depends	List of verb codes
		Depends	List of Data fields

Remarks:

CBMRLOCA: when both bits are off the CBMR handler will not allocate a work area, rather it will use CBMRWORK/CBMRWLEN instead.

CBMRVRBS: The number of verbs in the body of the CBMR. This number is always a multiple of 4.

CBMRSIZE: length in bytes of CBMR header and tail combined. Tail follows header directly

CBMR description – body for ACB

The CBMR header is described in the addenda [here](#)

The CBMR tail for ACB is applicable only when the CBMR header's CBMRACB bit is on.

It's length is determined by the CBMRVRBS fields in the CBMR header. This is always a multiple of 4 to allow word alignment of the tail area that directly follows the body section.

The body is directly followed by the tail, if applicable. It contains a data field of 4 or 8 bytes for each verb coded in the body, in the same sequence. The starting point of the tail can be found by adding the CBMRVRBS value to the end of the CBMR header. Its length can be deduced from the CBMRSIZE field, by subtracting both the header length and the CBMRVRBS field.

Label	Equate	Designation	Remarks
CBMR		DSECT	
CBMRBODY			
	CBMRACB_ESDS	X'20'	Cluster is ESDS
	CBMRACB_KSDS	X'21'	Cluster is KSDS
	CBMRACB_LDS	X'22'	Cluster is LDS
	CBMRACB_RRDS	X'23'	Cluster is RRDS
	CBMRACB_REPL	X'24'	Index is replicated
	CBMRACB_SPAN	X'25'	Spanned
	CBMRACB_SSWD	X'26'	Sequence set with data
	CBMRACB_VRRDS	X'27'	Variable RRDS
	CBMRACB_WCK	X'28'	Write checks
	CBMRACB_CMPRS	X'29'	Compression
	CBMRACB_UNQ	X'2A'	Unique key
	CBMRACB_XADDR	X'2B'	8-byte RBAs
	CBMRACB_OPEN	X'2C'	ACB is open
	CBMRACB_PATH	X'2D'	ACB uses a path
	CBMRACB_BASE	X'2E'	ACB uses base cluster
	CBMRACB_AIX	X'2F'	ACB uses AIX
	CBMRACB_VESDS	X'30'	Variable ESDS
	CBMRACB_COPIES	X'60'	Nr of copies
	CBMRACB_ERET	X'61'	Error return address
	CBMRACB_PASSWD	X'62'	Password pointer
	CBMRACB_EXLST	X'63'	EXLST pointer
	CBMRACB_BUFSP	X'64'	Buffer space
	CBMRACB_BUFND	X'65'	Nr of data buffers
	CBMRACB_BUFNI	X'66'	Nr of index buffers
	CBMRACB_STRNO	X'67'	Parallel threads
	CBMRACB_BSTRNO	X'68'	Nr of strings

Label	Equate	Designation	Remarks
CBMR		DSECT	
	CBMRACB_MACRF	X'69'	MACRF parms
	CBMRACB_RMODE31	X'6A'	31-bit buffers
	CBMRACB_SHRPL	X'6C'	Sharepool number
	CBMRACB_ACBLEN	X'6D'	Length of ACB
	CBMRACB_AVSPAC	X'6E'	Available space
	CBMRACB_BFRND	X'6F'	Buffer found count
	CBMRACB_BUFNO	X'70'	Nr of I/O buffers
	CBMRACB_BUFRDS	X'71'	Buffer read count
	CBMRACB_CINV	X'72'	Block size
	CBMRACB_ENDRBA	X'73'	End RBA
	CBMRACB_ERROR	X'74'	Open/close error code
	CBMRACB_FS	X'75'	Free Space
	CBMRACB_HALCRBA	X'76'	High allocated RBA
	CBMRACB_KEYLEN	X'77'	Key length
	CBMRACB_LRECL	X'78'	(max) record length
	CBMRACB_NCIS	X'79'	Nr of Block splits
	CBMRACB_NDELRL	X'7A'	Nr. of deletes
	CBMRACB_NEXCP	X'7B'	Nr of I/O requests
	CBMRACB_NEXT	X'7C'	Nr. of file extents
	CBMRACB_NINSR	X'7D'	Nr of inserts
	CBMRACB_NIXL	X'7E'	Nr of index levels
	CBMRACB_NLOGR	X'7F'	Nr of records
	CBMRACB_NRETR	X'80'	Nr of retrievals
	CBMRACB_NSSS	X'81'	Nr of CA splits
	CBMRACB_NUIW	X'82'	Nr of VSAM writes
	CBMRACB_NUPDR	X'83'	Nr of updates
	CBMRACB_RKP	X'84'	Relative key position
	CBMRACB_SDTASZ	X'85'	Uncompressed data size
	CBMRACB_STRMAX	X'86'	Max. nr of strings
	CBMRACB_UIW	X'87'	User initiated writes
	CBMRACB_DDNM	X'E0'	DDNAME
	CBMRACB_CDTASZ	X'E1'	Compressed data size
	CBMRACB_LEVEL	X'E2'	zVSAM level
	CBMRACB_LOKEY	X'E3'	Lowest valid key pointer
	CBMRACB_RELEASE	X'E4'	zVSAM level

Label	Equate	Designation	Remarks
CBMR		DSECT	
	CBMRACB_STMST	X'E5'	Last close timestamp
	CBMRACB_XAVCSPC	X'E6'	8-byte AVCSPAC
	CBMRACB_XENDRBA	X'E7'	8-byte ENDRBA
	CBMRACB_XHALCRBA	X'E8'	8-byte HALXRBA

The validity of verb codes differ per macro. Whether any given verb code is valid is determined by the syntax definition of the GENCB, MODCB, SHOWCB, TESTCB macros. Please refer to the appropriate chapter to check which parameters are supported.

For SHOWCB and TESTCB some keywords are applicable only after open. If they are issued before the ACB has been opened, an error is returned.

Additional notes:

CBMRACB_NRS – TESTCB only, always true

CBMRACB_RST – TESTCB only, always false

CBMRACB_NSR – TESTCB only, always true

CBMRACB_LSR – TESTCB only, always false

CBMRACB_GSR – TESTCB only, always false

CBMRACB_RLS – TESTCB only, always false

CBMRACB_NUB – TESTCB only, always true

CBMRACB_UBF – TESTCB only, always false

CBMRACB_CFX – TESTCB only, always false

CBMRACB_NFX – TESTCB only, always false

CBMRACB_DDN – TESTCB only, always false

CBMRACB_DSN – TESTCB only, always false

CBMRACB_ICI – TESTCB only, always false

CBMRACB_NCI – TESTCB only, always true

CBMRACB_LEW – TESTCB only, always true

CBMRACB_NLW – TESTCB only, always false

CBMRACB_REPL – TESTCB only, always false

CBMRACB_SSWD – TESTCB only, always false

CBMRACB_WCK – TESTCB only, always false

CBMRACB_CMPRS – TESTCB only, always false

CBMRACB_XADDR – TESTCB only, always true

CBMRACB_COPIES – GENCB only. If not specified, the CBMR handler assumes 1.

CBMRACB_PASSWD – pointer to a one-byte length field, followed by the password

CBMRACB_MAREA – TESTCB only, always 0

CBMRACB_MLEN – TESTCB only, always 0

CBMRACB_SHRPL – TESTCB only, always 0

CBMRACB_ENDRBA – ending RBA of the component, derived from ending XLRA.

CBMRACB_FS – Nr of free blocks per 100

CBMRACB_HALCRBA – High allocated RBA, derived from highest allocated XLRA.

CBMRACB_NEXT – for zVSAM the value is always 1.

CBMRACB_NSSS – always 0

CBMRACB_LEVEL – 4-byte address followed by 4-byte length of level info field

CBMRACB_LOKEY – 4-byte address followed by 4-byte length of key field

CBMRACB_RELEASE – 4-byte address followed by 4-byte length of level info field

CBMR description – body for EXLST

The CBMR header is described in the addenda [here](#)

The CBMR tail for EXLST is applicable only when the CBMR header's CBMRXLST bit is on.

It's length is determined by the CBMRVRBS fields in the CBMR header. This is always a multiple of 4 to allow word alignment of the tail area that directly follows the body section.

The body is directly followed by the tail, if applicable. It contains a data field of 4 or 8 bytes for each verb coded in the body, in the same sequence. The starting point of the tail can be found by adding the CBMRVRBS value to the end of the CBMR header. Its length can be deduced from the CBMRSIZE field, by subtracting both the header length and the CBMRVRBS field.

Label	Equate	Designation	Remarks
CBMR		DSECT	
CBMRBODY			
CBMRXL_VERBS		XL1	Verb code, repeats
	CBMRXL_NOOP	X'00'	Filler, no-operation
	CBMRXL_COPIES	X'60'	Nr of copies
	CBMRXL_ERET	X'61'	Error return address
	CBMRXL_XLSTLEN	X'6D'	Length of EXLST
	CBMRXL_NEODAD	X'A0'	No EODAD routine
	CBMRXL_EODAD	X'A1'	End-of-data routine
	CBMRXL_NEOD_A	X'A2'	(,A)
	CBMRXL_EOD_A	X'A3'	(address,A)
	CBMRXL_NEOD_N	X'A4'	(,N)
	CBMRXL_EOD_N	X'A5'	(address,N)
	CBMRXL_NEOD_L	X'A8'	(,L)
	CBMRXL_EOD_L	X'A9'	(address,L)
	CBMRXL_NEOD_AL	X'AA'	(,A,L)
	CBMRXL_EOD_AL	X'AB'	(address,A,L)
	CBMRXL_NEOD_NL	X'AC'	(,N,L)
	CBMRXL_EOD_NL	X'AD'	(address,N,L)
	CBMRXL_NLERAD	X'B0'	No LERAD routine
	CBMRXL_LERAD	X'B1'	Logical error routine
	CBMRXL_NLER_A	X'B2'	(,A)
	CBMRXL_LER_A	X'B3'	(address,A)
	CBMRXL_NLER_N	X'B4'	(,N)
	CBMRXL_LER_N	X'B5'	(address,N)
	CBMRXL_NLER_L	X'B8'	(,L)
	CBMRXL_LER_L	X'B9'	(address,L)
	CBMRXL_NLER_AL	X'BA'	(,A,L)

Label	Equate	Designation	Remarks
CBMR		DSECT	
	CBMRXL_LER_AL	X'BB'	(address,A,L)
	CBMRXL_NLER_NL	X'BC'	(,N,L)
	CBMRXL_LER_NL	X'BD'	(address,N,L)
	CBMRXL_NSYNAD	X'C0'	No SYNAD routine
	CBMRXL_SYNAD	X'C1'	Physical error routine
	CBMRXL_NSYN_A	X'C2'	(,A)
	CBMRXL_SYN_A	X'C3'	(address,A)
	CBMRXL_NSYN_N	X'C4'	(,N)
	CBMRXL_SYN_N	X'C5'	(address,N)
	CBMRXL_NSYN_L	X'C8'	(,L)
	CBMRXL_SYN_L	X'C9'	(address,L)
	CBMRXL_NSYN_AL	X'CA'	(,A,L)
	CBMRXL_SYN_AL	X'CB'	(address,A,L)
	CBMRXL_NSYN_NL	X'CC'	(,N,L)
	CBMRXL_SYN_NL	X'CD'	(address,N,L)
	CBMRXL_NJRNAD	X'D0'	No JRNAD routine
	CBMRXL_JRNAD	X'D1'	Journaling routine
	CBMRXL_NJRN_A	X'D2'	(,A)
	CBMRXL_JRN_A	X'D3'	(address,A)
	CBMRXL_NJRN_N	X'D4'	(,N)
	CBMRXL_JRN_N	X'D5'	(address,N)
	CBMRXL_NJRN_L	X'D8'	(,L)
	CBMRXL_JRN_L	X'D9'	(address,L)
	CBMRXL_NJRN_AL	X'DA'	(,A,L)
	CBMRXL_JRN_AL	X'DB'	(address,A,L)
	CBMRXL_NJRN_NL	X'DC'	(,N,L)
	CBMRXL_JRN_NL	X'DD'	(address,N,L)

The validity of verb codes differ per macro. Whether any given verb code is valid is determined by the syntax definition of the GENCB, MODCB, SHOWCB, TESTCB macros. Please refer to the appropriate chapter to check which parameters are supported.

API for RPL-based interfaces

RPL macro parameters

All supported parameters are implemented compatibly with IBM's VSAM implementation. For details, please refer to the relevant IBM manual.

For ease of access a short summary follows here:

AM=	Optional parameter. AM=VSAM is the default. No other values are supported.
ACB=	Pointer to an open ACB that represents the cluster to be accessed
AREA=	Record area. In Move mode reading a record implies moving the record into this area. In locate mode a pointer to the record is moved into the area instead.
AREALEN=	Length of record area
ARG=	Pointer to search argument. This is a key, a relative record number, or an RBA.
KEYLEN=	Length of key value specified in ARG when a generic key search is requested
ECB=	Pointer to ECB. Used with Asynchronous requests.
MSGAREA=	Pointer to a message area where error information may be returned
MSGLEN=	Length of message area
NXTRPL=	Pointer to next RPL in the chain. RPLs can be chained together to request a series of operations in a single call to zVSAM.
OPTCD=	List of keywords specifying how the request is to be handled. Please see below for the list of supported keywords and their meaning.
RECLEN=	Record length. Required when updating or adding records. When updating a record that has not changed its length, the parameter can be omitted if the immediately preceding operation on the RPL was the read for the record being updated.
ADR KEY	Mutually exclusive OPTCD keywords indicating whether the cluster is to be accessed by address or by key. ADR can be used with KSDS or ESDS to access records by RBA or XRBA. KEY can be used with KSDS to access records by key KEY can be used with RRDS to access records by relative record number.
DIR	OPTCD keyword indicating that the cluster is to be processed directly. DIR can be used with KSDS, ESDS or RRDS to access data randomly
SEQ	OPTCD keyword indicating that the cluster is to be processed sequentially. SEQ can be used with KSDS, ESDS or RRDS to access data sequentially SEQ serves as a default among the subgroup DIR SEQ SKP, which are otherwise independent OPTCD keywords.

SKP	OPTCD keyword to request skip-sequential access. SKP can be used with KSDS to randomly position the file to a specific key prior to sequential access SKP can be used with RRDS to randomly position the file to a specific RRN prior to sequential access
ARD LRD	Mutually exclusive OPTCD keywords indicating whether the cluster is to be accessed as specified by the user/current position, or just the very last record.
FWD BWD	Mutually exclusive OPTCD keywords indicating whether the cluster is to be accessed in a forward or backward direction
SYN ASY	Synchronous or asynchronous handling of the request.
NUP UPD NSP	Mutually exclusive OPTCD keywords indicating whether or not the record is to be locked for update. Updates are allowed if the cluster was opened with the OUT option specified. NSP is used only with OPTCD=DIR to retain file position, to enable sequential access from this point.
KEQ KGE	Mutually exclusive OPTCD keywords indicating whether the key has to match exactly (KEQ) or – if an exact match cannot be found – the next higher value is acceptable too.
FKS GEN	Mutually exclusive OPTCD keywords indicating whether we're doing a full key search, or we're specifying only a partial key value. In the latter case KEYLEN has to be specified to indicate how many key positions we're specifying.
LOC MVE	Mutually exclusive OPTCD keywords indicating whether we're working in move mode (zVSAM moves record between user record buffer and zVSAM buffer) or locate mode (record is not moved, data are processed in the zVSAM buffer, zVSAM provides a pointer only)
RBA XRBA	Mutually exclusive OPTCD keywords indicating whether we're using 4-byte RBA values or 8-byte extended RBA values.

GENCB RPL macro parameters

All supported parameters are implemented compatibly with IBM's VSAM implementation. For details, please refer to the relevant IBM manual.

For ease of access a short summary follows here:

RPL=addr required to indicate the RPL-supporting logic of the macro is being invoked

COPIES= Number of identical copies to generate. The default is 1.
Specify a number between 1 and 65535

WAREA= Address of a work area where the RPL/RPLs is/are to be constructed.
When WAREA is specified, LENGTH must be specified too.
When WAREA is not specified, the CBMR handler allocates an area of storage.
The address of this area is returned in R1; its length in R0.

LENGTH= Length in bytes of the area indicated by WAREA.
When LENGTH is specified, WAREA must be specified as well.

LOC= Where a work area for constructing the RPL/RPLs is to be allocated.
Used only when WAREA and LENGTH are not specified.
Supports the keywords BELOW and ANY, with BELOW being the default
BELOW the work area is to be allocated below the line
ANY the work area is to be allocated above the line if possible, below the line otherwise.

other Any parameters and/or keywords supported by the RPL macro.
Please see the description of the RPL macro for details [here](#)
Supported parameters and keywords on the RPL macro are supported on GENCB RPL as well. Likewise, unsupported parameters and keywords on the RPL macro are not supported on GENCB RPL either.
How the parameters can be specified differs per parameter. For a complete list of options, please see the IBM manual "DFSMS Macro Instructions for Data Sets" or equivalent for the operating system and version that you are porting to/from.
Please note: not supported are expressions like (S,scon) or (*,scon)

MF=I or omitted Specifies the standard form of the GENCB to generate an inline CBMR and an inline call to the CBMR handler.

MF=L Specifies the list form of the GENCB macro which generates an inline CBMR but no call to the CBMR handler.

MF=(L,addr) Specifies the list form of the GENCB macro to generate a remote CBMR at the indicated location. No call to the CBMR handler is generated.

MF=(L,addr,label) Same as MF=(L,addr) but label will be equated to the length of the CBMR.

MF=(E,addr) Specifies the execute form of the GENCB macro to generate code that will dynamically modify the CBMR at the indicated address according to the parameters specified before calling the CBMR handler.

MF=(G,addr) Specifies the generate form of the GENCB macro to generates code to modify the indicated CBMR as specified by the other parameters and to call the CBMR handler.

MF=(G,addr,label)

Same as MF=(G,addr) but label will be equated to the length of the CBMR

MODCB RPL macro parameters

All supported parameters are implemented compatibly with IBM's VSAM implementation. For details, please refer to the relevant IBM manual.

For ease of access a short summary follows here:

RPL=addr required to indicate the RPL to be modified

other Any parameters and/or keywords supported by the RPL macro.
Please see the description of the RPL macro for details [here](#)
Supported parameters and keywords on the RPL macro are supported on MODCB RPL as well. Likewise, unsupported parameters and keywords on the RPL macro are not supported on MODCB RPL either.
How the parameters can be specified differs per parameter. For a complete list of options, please see the IBM manual “DFSMS Macro Instructions for Data Sets” or equivalent for the operating system and version that you are porting to/from.
Please note: not supported are expressions like (S,scon) or (*,scon)

MF=I or omitted

Specifies the standard form of the MODCB to generate an inline CBMR and an inline call to the CBMR handler.

MF=L

Specifies the list form of the MODCB macro which generates an inline CBMR but no call to the CBMR handler.

MF=(L,addr) Specifies the list form of the MODCB macro to generate a remote CBMR at the indicated location. No call to the CBMR handler is generated.

MF=(L,addr,label)

Same as MF=(L,addr) but label will be equated to the length of the CBMR.

MF=(E,addr) Specifies the execute form of the MODCB macro to generate code that will dynamically modify the CBMR at the indicated address according to the parameters specified before calling the CBMR handler.

MF=(G,addr) Specifies the generate form of the MODCB macro to generate code to modify the indicated CBMR as specified by the other parameters and to call the CBMR handler.

MF=(G,addr,label)

Same as MF=(G,addr) but label will be equated to the length of the CBMR

SHOWCB RPL macro parameters

All supported parameters are implemented compatibly with IBM's VSAM implementation. For details, please refer to the relevant IBM manual.

For ease of access a short summary follows here:

RPL=addr required to indicate the RPL to be queried

Each keyword specified returns a field of 4 or 8 bytes. These return values are stored consecutively in the return area specified in the AREA= and LENGTH= parameters. Some keywords are valid only when the RPL's request is not in progress. Unpredictable results may occur when any of these keywords are used on a SHOWCB RPL request that is still processing.

For some keywords there are a few things to keep in mind:

FIELDS=RBA/XRBA – zVSAM supports these keywords only for ESDS. For any other type of cluster a value of foxes will be returned by default.

TRANSID – Always returns foxes.

MF=I or omitted

Specifies the standard form of the SHOWCB to generate an inline CBMR and an inline call to the CBMR handler.

MF=L

Specifies the list form of the SHOWCB macro which generates an inline CBMR but no call to the CBMR handler.

MF=(L,addr)

Specifies the list form of the SHOWCB macro to generate a remote CBMR at the indicated location. No call to the CBMR handler is generated.

MF=(L,addr,label)

Same as MF=(L,addr) but label will be equated to the length of the CBMR.

MF=(E,addr)

Specifies the execute form of the SHOWCB macro to generate code that will dynamically modify the CBMR at the indicated address according to the parameters specified before calling the CBMR handler.

MF=(G,addr)

Specifies the generate form of the SHOWCB macro to generate code to modify the indicated CBMR as specified by the other parameters and to call the CBMR handler.

MF=(G,addr,label)

Same as MF=(G,addr) but label will be equated to the length of the CBMR

TESTCB RPL macro parameters

All supported parameters are implemented compatibly with IBM's VSAM implementation. For details, please refer to the relevant IBM manual.

For ease of access a short summary follows here:

RPL=addr required to indicate the RPL to be tested

FTNCD=nr Values used for FTNCD and their meaning can be found in the IBM manual “DFSMS Macro Instructions for Datasets”, chapter “Return and Reason Codes”, section “Component Codes”

RBA=nr – zVSAM supports this keyword only for ESDS. For any other type of cluster a value of foxes will be assumed by default.

MF=I or omitted

Specifies the standard form of the TESTCB to generate an inline CBMR and an inline call to the CBMR handler.

MF=L Specifies the list form of the TESTCB macro which generates an inline CBMR but no call to the CBMR handler.

MF=(L,addr) Specifies the list form of the TESTCB macro to generate a remote CBMR at the indicated location. No call to the CBMR handler is generated.

MF=(L,addr,label)

Same as MF=(L,addr) but label will be equated to the length of the CBMR.

MF=(E,addr) Specifies the execute form of the TESTCB macro to generate code that will dynamically modify the CBMR at the indicated address according to the parameters specified before calling the CBMR handler.

MF=(G,addr) Specifies the generate form of the TESTCB macro to generates code to modify the indicated CBMR as specified by the other parameters and to call the CBMR handler.

MF=(G,addr,label)

Same as MF=(G,addr) but label will be equated to the length of the CBMR

POINT macro parameters

GET macro parameters

PUT macro parameters

ERASE macro parameters

CHECK macro parameters

ENDREQ macro parameters

VERIFY macro parameters

zRPL description

The structure and layout of the zRPL are not formally part of the interface and may change in future releases. Therefore the zRPL layout is shown here only for the sake of completeness. Direct access to subfields in the zRPL is discouraged. Use SHOWCB RPL, TESTCB RPL and/or MODCB RPL to inspect, test, and/or modify the zRPL's content. Accessing subfields of the zRPL directly may adversely impact portability of your programs.

Label	Equate	Designation	Remarks
IFGRPL IHARPL		DSECT	
RPLEYE		CL4	Eye catcher
	RPLZRPL	C'zRPL'	Fixed value
RPLACB		AL4	Pointer to ACB
RPLAREA		AL4	Pointer to record area
RPLAREAL		XL4	Length of record area
RPLARG		AL4	Pointer to argument
RPLECB		AL4	Pointer to ECB
RPLMSGAR		AL4	Pointer to message area
RPLRPL		AL4	Pointer to chained RPL
RPLRECLN		XL4	Length of record
RPLMSGLN		XL2	Length of message area
RPLKEYLN		XL1	Key length
		XL1	Reserved
RPLOPTCD		0XL2	Option codes
RPLOPTCD1		XL1	Option byte 1
	RPLOPT_KEY	X'80'	1: OPTCD=KEY 0: OPTCD=ADR
	RPLOPT_SEQ	X'40'	1: OPTCD=SEQ 0: OPTCD=DIR
	RPLOPT_SKP	X'20'	1: OPTCD=SKP 0: OPTCD=SEQ/DIR
	RPLOPT_ARD	X'10'	1: OPTCD=ARD 0: OPTCD=LRD
	RPLOPT_FWD	X'08'	1: OPTCD=FWD 0: OPTCD=BWD
	RPLOPT_SYN	X'04'	1: OPTCD=SYN 0: OPTCD=ASY
	RPLOPT_NUP	X'02'	1: OPTCD=NUP 0: OPTCD=UPD
	RPLOPT_NSP	X'01'	1: OPTCD=NSP 0: OPTCD=NUP/UPD

Label	Equate	Designation	Remarks
IFGRPL IHARPL		DSECT	
RPLOPTCD2		XL1	Option byte 2
	RPLOPT_KEQ	X'80'	1: OPTCD=KEQ 0: OPTCD=KGE
	RPLOPT_FKS	X'40'	1: OPTCD=FKS 0: OPTCD=GEN
	RPLOPT_MVE	X'20'	1: OPTCD=MVE 0: OPTCD=LOC
	RPLOPT_RBA	X'10'	1: OPTCD=RBA 0: OPTCD=XRBA
		XL6	Reserved

CBMR description – body for RPL

The CBMR header is described in the addenda [here](#)

The CBMR tail for RPL is applicable only when the CBMR header's CBMRRPL bit is on.

It's length is determined by the CBMRVRBS fields in the CBMR header. This is always a multiple of 4 to allow word alignment of the tail area that directly follows the body section.

The body is directly followed by the tail, if applicable. It contains a data field of 4 or 8 bytes for each verb code in the body, in the same sequence. The starting point of the tail can be found by adding the CBMRVRBS value to the end of the CBMR header. Its length can be deduced from the CBMRSIZE field, by subtracting both the header length and the CBMRVRBS field.

Label	Equate	Designation	Remarks
CBMR		DSECT	
CBMRBODY			
	CBMRRPL_PKP	X'3F'	AIXFLAG=AIXPKP
	CBMRRPL_COMPL	X'40'	IO=COMPLETE
	CBMRRPL_COPIES	X'60'	Nr of copies
	CBMRRPL_ERET	X'61'	Error return address
	CBMRRPL_OPTCD	X'68'	OPTCD Parms
	CBMRRPL_MAREA	X'69'	Message area
	CBMRRPL_MLEN	X'6A'	Message area length
	CBMRRPL_RPLEN	X'6D'	Length of RPL
	CBMRRPL_KEYLEN	X'77'	Key length
	CBMRRPL_RECLN	X'78'	record length
	CBMRRPL_ACB	X'90'	ACB pointer
	CBMRRPL_AREA	X'91'	Record area
	CBMRRPL_AREALEN	X'92'	Area length
	CBMRRPL_ARG	X'93'	Argument pointer
	CBMRRPL_ECB	X'94'	ECB pointer
	CBMRRPL_NXTRPL	X'95'	Next RPL in chain
	CBMRRPL_AIXPC	X'97'	Nr of AIX pointers
	CBMRRPL_FDBK	X'98'	Feedback code
	CBMRRPL_FTNCN	X'99'	Function code
	CBMRRPL_CRBA	X'9A'	Current RBA
	CBMRTPL_XRBA	X'F0'	8-byte Current RBA

The validity of verb codes differ per macro. Whether any given verb code is valid is determined by the syntax definition of the GENCB, MODCB, SHOWCB, TESTCB macros. Please refer to the appropriate chapter to check which parameters are supported.

For SHOWCB and TESTCB some keywords are applicable only when no I/O is in progress. If they are issued before the ECB has been posted, unpredictable results may occur.

Additional notes:

CBMRRPL_CNV – TESTCB only, always false

CBMRRPL_TRANSID – Always foxes

List of changes

Date	Author	Description
2018-09-16	Abe Kornelis	Remove SPX from VS records that have only a single segment. Change order and numbering of chapters Move macro parameter descriptions to addendum Expand chapter on compatibility In the addendum for GENCB ACB add explanation on MF usage
2018-09-18	Abe Kornelis	Various small changes as suggested by Hugh Sweeney Moved zACB and zEXLST layout paragraphs to the addenda.
2018-09-20	Abe Kornelis	Various small changes as suggested by Melvyn. See mail dated 2018-09-19 22:19
2018-09-27	Abe Kornelis	Added content for MODCB ACB, including addendum.
2018-09-29	Abe Kornelis	Added content for SHOWCB ACB, including addendum
2018-10-01	Abe Kornelis	Added content for TESTCB ACB, including addendum
2018-10-07	Abe Kornelis	Added comment on CBMR layout to chapters on GENCB ACB, MODCB ACB, SHOWCB ACB and TESTCB ACB. Parm AM=VSAM added to GENCB ACB chapter. Added content for GENCB EXLST, including addendum
2018-10-08	Abe Kornelis	Added content for MODCB EXLST, including addendum Added content for SHOWCB EXLST, including addendum Added content for TESTCB EXLST, including addendum
2018-10-09	Abe Kornelis	CBMR split into header and separate tail sections CBMR header description added
2018-10-10	Abe Kornelis	Minor changes as suggested by Melvyn's mail dd 2018-10-09 23:40 Addition of chapter titles for RPL-based interfaces to addenda.
2018-10-11	Abe Kornelis	Added CBMR description – body for ACB
2018-10-13	Abe Kornelis	Added CBMR description – body for EXLST Added RPL macro description, including addendum Added GENCB RPL macro description, including addendum Added MODCB RPL macro description, including addendum Added SHOWCB RPL macro description, including addendum Added TESTCB RPL macro description, including addendum
2018-10-15	Abe Kornelis	Added ACBPFX pointer to zACB layout
2018-10-16	Abe Kornelis	Added CBMR description – body for RPL ACBTYPE → ACBSTYPE Removed ACBMACR3_NLW and ACBMACR3_MODE ACBCUEL → ACBUEL ACBOCK → ACBLOCK
2018-10-21	Abe Kornelis	ACB ADR/KEY improved keyword description in addendum ACB IN/OUT improved keyword description in addendum ACB DDNAME improved keyword description in ACB macro chapter and the addendum Unsupported parameters and keywords on ACB, EXLST, RPL changed from “flagged as error” to “ignored”

Date	Author	Description
2018-10-22	Abe Kornelis	Add description of prefix block, including counters area. Updated addendum for SHOWCB/TESTCB with reference to source of data for each keyword. Added prefix field PFXIXLVL. Added instructions for RPL-based operations on how to maintain prefix counter fields. Added description of spacemap block.
2018-10-23	Abe Kornelis	Specify that SHOWCB/TESTCB for RBA/XRBA is supported for ESDS only. Foxes for any other component.
2018-10-24	Abe Kornelis	Add description for block header, block, footer, record pointer list
2018-10-26	Abe Kornelis	Added description of open macro logic
2018-10-27	Abe Kornelis	Added eyecatcher to the prefix area, moved record length and key info fields to beginning of prefix area BHDRPREV/NEXT on prefix block documented as being foxes
2018-10-28	Abe Kornelis	Added ACBVER to zACB layout Added Area to Terminology chapter Max. block size reduced from 2G to 16MB Added ACBXPFX to zACB layout Added description of open execution logic
2018-11-21	Abe Kornelis	In API on macro interfaces improved wording for handling of (as yet) unsupported macro parameters. Add ATRB=VESDS for TESTCB ACB Improved picture and text on spacemap block layout
2018-11-22	Abe Kornelis	BHDRPREV/NEXT details expanded Removed PFXBSEG/PFXESEG
2018-11-25	Abe Kornelis	Spacemap area structure. Segmented records were missing. Added.
2018-11-29	Abe Kornelis	Added diagrams to chapter on block header structure.
2018-12-02	Abe Kornelis	Added alternative diagram for chaining segments. Preferred solution not yet determined And added drawings for chaining index blocks
2018-12-09	Abe Kornelis	Structure of Physical files: ELIX added to the list of block types Block Header Structure: BHDRFLGS changed to BHDRFLG1 and added BHDRFLG2 with BHDR_ELX
2018-12-17	Abe Kornelis	Put PFXBSEG/PFXESEG back in Corrected typos in drawings for explaining BHDRNEXT/PREV RPTR_END no longer all foxes, foxes only for RPTRREC@ Added 4 date fields to the prefix structure for creation and last update timestamps for both data and index component. MF=omitted changed to MF= in various locations
2019-01-06	Abe Kornelis	Added various fields to RPL

Date	Author	Description
2019-01-23	Melvyn Maltz	Took over the D&L, not documenting spelling, syntax, bad references and other trivia Added hyperlinks Amended CBMRACB fields Amended CBMRRPL fields Removed RPL TIMEOUT...VTAM only Updated Prefix Block DSECT